Jordan Arnold

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Tabbawala 360 | Interactive Embodiment Research</title>
  <meta name="description" content="Tabbawala 360 — Dance and Charlie cut bamboo together" />

  <link rel="preconnect" href="https://fonts.gstatic.com" />
  <link rel="stylesheet" href="https://fonts.google.com" onload="void" />
  <noscript><link rel="stylesheet" href="https://fonts.google.com" />
  <meta name="theme-color" content="#0d0d0d" />

  <style>
    /* ═══ ═══
    :root {
      --primary-gold: #D4AF37;
      --primary-gold-light: #F0D060;
      --primary-gold-dark: #B8962E;
      --bg-deep: #0a0a0a;
      --bg-card: #1a1a1a;
      --bg-light: #F7F5E8;
      --text-dark: #1a1a1a;
      --text-gray: #666666;
      --text-light: #F7F5E8;
      --text-muted: #a1a1a1;
      --glass-bg: rgba(26, 26, 26, 0.7);
      --glass-bg-light: rgba(255, 255, 255, 0.7);
      --glass-bg-dark: rgba(0, 0, 0, 0.5);
      --glass-gold: #D4AF37;
      --shadow-sm:
        0 1px 2px rgba(0, 0, 0, 0.5),
        0 0 0 1px rgba(212, 175, 55, 0.1);
      --shadow-md:
        0 4px 6px rgba(0, 0, 0, 0.5),
        0 0 0 1px rgba(212, 175, 55, 0.2);
      --shadow-lg:
        0 10px 15px rgba(0, 0, 0, 0.5),
        0 0 0 1px rgba(212, 175, 55, 0.3);
      --shadow-gold: 0 0 20px rgba(212, 175, 55, 0.3);
      --shadow-gold-strong: 0 0 40px rgba(212, 175, 55, 0.5);
      --gradient-gold: linear-gradient(135deg, #D4AF37 0%, #F0D060 50%, #B8962E 100%);
      --gradient-dark: linear-gradient(135deg, #1a1a1a 0%, #0a0a0a 100%);
      --gradient-hero: linear-gradient(135deg, rgba(26, 26, 26, 0.9) 0%, rgba(10, 10, 10, 0.8) 100%);
      --font-sans: "Inter", "Noto Sans", sans-serif;
      --font-mono: "Fira Code", "Fira Mono", monospace;
      --font-size: "Noto Sans Bengali", sans-serif;
      --font-origin: "Noto Sans Bengali", sans-serif;

      --section-padding: 100px;
      --container-max: 1200px;

      --radius-xsf: 0 0px 0px rgba(0, 0, 0, 0.);
      --radius-gold: 0 0px 0px rgba(212, 175, 55, 0.);
      --radius-gold-strong: 0 0px 0px rgba(212, 175, 55, 0.);
    }

    /* Prefers-reduced-motion: disable all animations and transitions */
    @media (prefers-reduced-motion: reduce) {
      *,
      *::before,
      *::after {
        animation-duration: 0.01ms !important;
        animation-iteration-count: 1 !important;
        transition-duration: 0.01ms !important;
        scroll-behavior: auto;
      }
    }

    /* ═══ ═══
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    html {
      font-family: var(--font-sans);
      scroll-behavior: smooth;
      -webkit-text-size-adjust: 100%;
      -moz-text-size-adjust: 100%;
      text-size-adjust: 100%;
      -webkit-font-smoothing: ;
      -moz-osx-font-smoothing: grayscale;
      text-rendering: optimizeLegibility;
      overflow-x: hidden;
      overflow-y: scroll;
    }

    body {
      -webkit-scrollbar {width: 10px; }
      -webkit-scrollbar-track { background: var(--bg-deep); }
      -webkit-scrollbar-thumb { background: var(--gold-dark); border-radius: 5px; }
      -webkit-scrollbar-thumb:hover { background: var(--primary-gold-dark); }

      display:block flex { display:block flex;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1);opacity:1;transform:translateY(0px);-webkit-backdrop-filter:blur(0px);backdrop-filter:blur(0px); }
      display:block flex:hover { opacity:0.8;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1); }
      display:block flex:active { opacity:0.7;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1);transform:translateY(0.0px); }
      display:block inline-flex { display:block inline-flex;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1);-webkit-backdrop-filter:blur(0px);backdrop-filter:blur(0px);-webkit-text-fill-color:rgba(0,0,0,0.);-webkit-text-stroke-color:rgba(0,0,0,0.);-webkit-text-stroke-width:0.0px;-webkit-mask-image:linear-gradient(rgba(0,0,0,0.),rgba(0,0,0,0.);-webkit-mask-size:100% 100%;-webkit-mask-position:0.0% 0.0px;-webkit-mask-repeat:no-repeat;-webkit-mask-clip:content-box;-webkit-mask-composite:source-over;-webkit-mask-mode:alpha;-webkit-mask-origin:content-box
```

```
    .separation.photo.shot:hover { border-color: rgba(255,179,71,0.5); }
    .separation.photo.selected { border-color: var(--primary-gold); }
    .separation.photo.shot.selected { border-color: var(--primary-gold); }
    .separation.photo .photo-check { position: absolute; top: 8px; right: 8px; width: 40px; height: 40px; background: var(--primary-gold); border-radius: 50%; display: none; align-items: center; justify-content: center; color: var(--navy-dark); font-weight: bold; }
    .separation.photo.selected .photo-check { display: flex; }

    .separation.photo.photo-placeholder { display: flex; gap: 10px; overflow-x: auto; padding: 15px 0; min-height: 140px; background: rgba(0,0,0,0.3); border-radius: 8px; margin: 15px 0; }
    .separation.photo.photo-placeholder:hover { color: var(--primary-gold); }
    .separation.photo.photo-placeholder.shot { min-width: 80px; height: 80px; background: rgba(255,179,71,0.1); border: 1px solid rgba(255,179,71,0.3); }
    .separation.photo.photo-placeholder.shot:hover { transform: scale(1.05); }
    .separation.photo.photo-placeholder.shot.selected { background: rgba(255,179,71,0.2); }
    .separation.photo.photo-placeholder.shot.selected::before { content: "✓"; position: absolute; top: 5px; right: 5px; width: 20px; height: 20px; background: var(--primary-gold); color: var(--navy-dark); border-radius: 50%; display: flex; align-items: center; justify-content: center; font-size: 12px; }

    /* Action grid styles */
    .action.selectable { cursor: pointer; border: 2px dashed transparent; padding: 2px 0px; border-radius: 4px; transition: var(--transition-smooth); }
    .action.selectable:hover { background: rgba(255,179,71,0.1); border-color: rgba(255,179,71,0.3); }
    .action.selected { background: rgba(255,179,71,0.2); border: 2px solid var(--primary-gold); }
    .action.edit-input { background: rgba(255,179,71,0.1); border: 1px solid var(--primary-gold); color: var(--cream); text-right; padding: 2px 0px; border-radius: 4px; font-family: inherit; font-size: inherit; width: 100%; }
    </style>

    <style_ref_finish_historical_web_layout_pdf_fr_1
    .header_fixed_fixed_fixed_web_layout_pdf_fr_1 { z-index:dashboard; .wrt: transparent; }
    .header_fixed_fixed_fr_1 {
    .header_fixed_fixed_fr_1: background:rgba(0,0,0,0.0); { z-index:10000; display:none;
    .header_fixed_fixed_fr_1:
    .header_fixed_scanning_active{ display:block; }
    .header_fixed_fixed_fr_1:
    position:fixed; border-radius:0px; box-shadow:0,0,0,0:0px; rgba(0,0,0,0.0); { z-index:10000; position:absolute;top;
    transition:all 0.3s ease;
    }
    .header_fixed_nav{
    .header_fixed_fixed_fr_1:0px; width:calc(100%-12px); rgba(0,0,0,0.0); background:var(--navy_dark);
    border:1px solid var(--glass_border); .header_fixed_fixed_fr_1:0px; .padding:15px;
    .header_fixed_fixed_fr_1:0px; .padding:15px;
    }
    .header_fixed_fixed_fr_1:0px;.header_fixed_fixed_fr_1:0px; .header_fixed_fixed_fr_1:0px; font_family:var(--font_display);
    .header_fixed_fixed_fr_1:var(--text_secondary);.header_fixed_fixed_fr_1:0px;.header_fixed_fixed_fr_1:0px;
    .
```

```
    color: var(--neg-dark);
    border-radius: 10px;
    cursor: pointer;
    text-weight: 700;
    transition: all 0.2s cubic-bezier(0.4,0.0,0.1);

  .tagfilter,.dashed-tree { transform: scale(1.0); }

  /* Subpage */
  .subpage-header{
    display:flex;
    gap:16px;
    justify-content:space-between;
    flex-wrap:wrap;
    gap: 16px;
    margin-bottom: 16px;

  .subpage-title{
    display:flex;
    gap:12px;
    flex-wrap:wrap;
    flex-basis: var(--text-display);
    text-weight: 700;
    color: var(--primary-grad);

  .subpage-dashed{
    display:flex;
    gap:12px;
    flex-wrap:wrap;
    text-weight: 700;
    color: var(--dashed-grad);
    margin-bottom: 16px;

  .subpage-form{
    display:flex;
    gap:12px;
    flex-wrap:wrap;

  .subpage-form-row{
    display:grid;
    grid-template-columns: 1fr;
    gap: 16px;

  .subpage-form-actions{
    display:flex;
    gap:12px;
    flex-wrap:wrap;
    flex-basis: var(--text-display);
    justify-content:flex-end;

  .subpage-table-wrap{
    overflow-x: hidden;
    text-weight: 700;
    background: rgba(0,0,0,0.4);
    border-radius: 10px;
    gap: 16px;

  .subpage-tag{
    display:flex;
    gap:12px;
    flex-wrap:wrap;
    text-weight: 700;
    color: var(--text-dashed);
    text-weight: 700;
    text-weight: 700;
    text-weight: 700;
```

```
  ,'ucrt','ijg',sizeof(FMT/\*)\*\*\*
```

```
            <div class="form-group">
              <label>Default Source (ui)</label>
              <input type="text" class="form-control" id="inputDefaultSource" value="f" dir="f" dow="dd" />
            </div>
          </div>
          <div
            <div style="display:grid; grid-template-columns: 1fr; gap:10px;">
              <div
                <div style="color:var(--primary-gold); margin-bottom:0px;">Select Photos:</b>
                <input type="file" id="inputPhotos" />
              </div>
              <div
                <div style="color:var(--primary-gold); margin-bottom:0px;">Video Sequence:</b>
                <input type="file" id="inputVideoSequence" />
                <button class="ubbo-ok-ubbo-ok-ubbo-primary" style="width:100%; [illegible]" id="inputLoadSequenceFrames">Load Sequence Frames</button>
              </div>
            </div>
          </div>
        </div>
      </div>

      <!-- Composition Player -->
      <div id="inspectCompositionPlayerWrapper" [illegible]>
        <div id="inspectCompositionPlayerHeader" [illegible]>Composition Preview</div>
        <div id="inspectCompositionPlayerBodyWrapper" [illegible]>
        <div id="inspectCompositionPlayerBodyWrapper" [illegible]>
        <div id="inspectCompositionPlayerBodyWrapper" [illegible]>
        <div id="inspectCompositionPlayerBodyWrapper" [illegible]>
        <div id="inspectCompositionPlayerBodyWrapper" [illegible]>
          <canvas id="inspectCompositionPlayer" [illegible]></canvas>
        </div>
      </div>

      <!-- Batch Upload Modal -->
      <div id="batchUploadModalOverlay" [illegible]>
        <div id="batchUploadModalBodyWrapper" [illegible]>
        <div id="batchUploadModalBodyWrapper" [illegible]>
          <div style="color:var(--primary-gold); [illegible]">Batch Upload</b>
          <div style="color:var(--primary-gold); font-display[illegible]">Select Photos:</b>
            <input type="file" id="bbu_input" dir="f" dow="dd" />
          </div>
          <div
            <div
              <div
                <div
                  <select id="bbu_target" dir="f" dow="dd">
                    <option value="Tuesday">Tuesday</option>
                    <option value="Wednesday">Wednesday</option>
                    <option value="Thursday">Thursday</option>
                    <option value="Friday">Friday</option>
                    <option value="Saturday">Saturday</option>
                    <option value="Sunday">Sunday</option>
                    <option value="Monday">Monday</option>
                    <option value="Tuesday">Tuesday</option>
                    <option value="Wednesday">Wednesday</option>
                  </select>
                </div>
                <div
                  <div
                    <input type="text" id="bbu_input" dir="f" dow="dd" />
                  </div>
```

```
    if (passkeyCredentials.length>0\&\&!isAuthenticated\&\&!passkeyAutoAttempted\&\&!
    hasRegistrationCookie\&\&!_syncAuthStarted\&\&!_syncTokenRefreshing\&\&!
    _syncTokenRetryTimer\&\&!_syncTokenRetryAttempts\&\&!_syncTokenRetryExhausted\&\&!_syncTokenRetryCooldowns\&\&!_syncTokenRetryCooldownTimer\&\&!_syncTokenRetryCooldownExpiredAt\&\&!_syncTokenRetryCooldownDurationMs
```

```
/! ================================================================
/! WEEK OFFSET CHECK
/! ================================================================
function updateWeekOffset()
{ var weekOffsetInput = document.getElementById('checkout-week-offset');
  document.getElementById('WeekOffset').value = weekOffsetInput ? weekOffsetInput.value : '';
  document.getElementById('WeekOffset').value = document.getElementById('WeekOffset').value || '';
  document.getElementById('WeekOffset').value = document.getElementById('WeekOffset').value || '';
  document.getElementById('WeekOffset').disabled = false;
}

function closeWeekOffsetModal() {document.getElementById('WeekOffsetModal').classList.remove('active');}
function updateWeekOffsetPreviewFromUI() {var weekOffsetInputEl=document.getElementById('weekOffset');var weekOffset=weekOffsetInputEl?weekOffsetInputEl.value:'';var
preEl=document.getElementById('weekOffsetPreviewLine');if(!preEl)return;if(!weekOffset||weekOffset==0){preEl.textContent='';return;}var
parts=weekOffset.split(',').filter(function(p){return p.trim()!=='';});var
if(!parts.length){preEl.textContent='';return;}var
dateStrs=[];var today=new Date();today.setHours(0,0,0,0);for(var
i=0;i<parts.length;i++){var
offset=parseInt(parts[i].trim,10);if(isNaN(offset)||offset<1||offset>52)continue;var
d=new Date(today);d.setDate(d.getDate()+offset*7);var
dd=d.getDate();var
mm=(d.getMonth()+1);var
yyyy=d.getFullYear();dateStrs.push(dd+'/'+mm+'/'+yyyy);}preEl.textContent=dateStrs.length?'Will apply to: '+dateStrs.
function closeWeekOffsetModal() {document.getElementById('WeekOffsetModal').classList.remove('active');}
function updateWeekOffsetPreviewFromUI() {var weekOffsetInputEl=document.getElementById('weekOffset');var weekOffset=weekOffsetInputEl?weekOffsetInputEl.value:'';
var
```

```
    c: left:left + left.width / 2,
    y: top:top + top.height / 2,
    width: left.width / 2 + 1,
    height: top.height / 2 + 1,
    x1: left.left,
    y1: top.top,
    x2: c.x,
    y2: c.y,
    angle: Math.atan2(c.y - y1, c.x - x1)
  }
}

var cnt = 0;
var dots = [];
var ctx = null;
var lastFrame = 0;
function init(){
  var canvas, e;
  if (!document.body)
    return;
  canvas = document.createElement('canvas');
  canvas.id = 'mouse-trail';
  canvas.style.position = 'fixed';
  canvas.style.pointerEvents = 'none';
  canvas.style.zIndex = '9999';
  canvas.style.top = '0';
  canvas.style.left = '0';
  canvas.width = window.innerWidth;
  canvas.height = window.innerHeight;
  document.body.appendChild(canvas);
  ctx = canvas.getContext('2d');
  if (!ctx)
    return;
  e = document.addEventListener('mousemove', function(ev) {
    var now = Date.now();
    if (now - lastFrame < 16)
    return;
    lastFrame = now;
    var x = ev.clientX;
    var y = ev.clientY;
    var dx = (x - (dots.length > 0 ? dots[0].x : 0));
    var dy = (y - (dots.length > 0 ? dots[0].y : 0));
    var dist = Math.sqrt(dx * dx + dy * dy);
    var steps = Math.floor(dist / 4);
    if (steps > 20)
    steps = 20;
    for (var i = steps; i > 0; i--) {
    var t = i / steps;
    var px = x - dx * t;
    var py = y - dy * t;
    var dot = {
      x: px,
      y: py,
      life: 1.0,
      size: (1 - t) * 8 + 2
    };
    dots.push(dot);
    if (dots.length > 200)
    dots.shift();
    }
  }, true);
  function loop() {
    var i, dot, alpha, size, grad;
    if (!ctx)
    return;
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    for (i = 0; i < dots.length; i++) {
    dot = dots[i];
    dot.life -= 0.02;
    if (dot.life <= 0)
    continue;
    alpha = dot.life * 0.6;
    size = dot.size * dot.life;
    if (!size)
    size = 1;
    grad = ctx.createRadialGradient(dot.x, dot.y, 0, dot.x, dot.y, size);
    grad.addColorStop(0, 'rgba(255,255,255,' + alpha + ')');
    grad.addColorStop(1, 'rgba(255,255,255,0)');
    ctx.fillStyle = grad;
    ctx.beginPath();
    ctx.arc(dot.x, dot.y, size, 0, 6.283185307179586);
    ctx.fill();
    }
    if (dots.length > 0)
    while (dots[0].life <= 0)
    dots.shift();
    requestAnimationFrame(loop);
  }
  requestAnimationFrame(loop);
}
if (document.readyState == 'loading')
document.addEventListener('DOMContentLoaded', init);
else
init();

function checkCookiePrivacy(){
  var cookies = document.cookie.split('; ');
  var i = 0;
  var parts = null;
  var name = null;
  var value = null;
  var consent = null;
  var allowed = false;
  var c = null;
  var cat = null;
  var cats = null;
  var j = 0;
  var k = 0;
  var m = null;
  var n = null;
  var o = null;
  var p = null;
  var q = null;
  var r = null;
  var s = null;
  var t = null;
  var u = null;
  var v = null;
  var w = null;
  var x = null;
  var y = null;
  var z = null;
  var aa = null;
  var ab = null;
  var ac = null;
  var ad = null;
  var ae = null;
  var af = null;
  var ag = null;
  var ah = null;
  var ai = null;
  var aj = null;
  var ak = null;
  var al = null;
  var am = null;
  var an = null;
  var ao = null;
  var ap = null;
  var aq = null;
  var ar = null;
  var as = null;
  var at = null;
  var au = null;
  var av = null;
  var aw = null;
  var ax = null;
  var ay = null;
  var az = null;
  var ba = null;
  var bb = null;
  var bc = null;
  var bd = null;
  var be = null;
  var bf = null;
  var bg = null;
  var bh = null;
  var bi = null;
  var bj = null;
  var bk = null;
  var bl = null;
  var bm = null;
  var bn = null;
  var bo = null;
  var bp = null;
  var bq = null;
  var br = null;
  var bs = null;
  var bt = null;
  var bu = null;
  var bv = null;
  var bw = null;
  var bx = null;
  var by = null;
  var bz = null;
  var ca = null;
  var cb = null;
  var cc = null;
  var cd = null;
  var ce = null;
  var cf = null;
  var cg = null;
  var ch = null;
  var ci = null;
  var cj = null;
  var ck = null;
  var cl = null;
  var cm = null;
  var cn = null;
  var co = null;
  var cp = null;
  var cq = null;
  var cr = null;
  var cs = null;
  var ct = null;
  var cu = null;
  var cv = null;
  var cw = null;
  var cx = null;
  var cy = null;
  var cz = null;
  var da = null;
  var db = null;
  var dc = null;
  var dd = null;
  var de = null;
  var df = null;
  var dg = null;
  var dh = null;
  var di = null;
  var dj = null;
  var dk = null;
  var dl = null;
  var dm = null;
  var dn = null;
  var do = null;
  var dp = null;
  var dq = null;
  var dr = null;
  var ds = null;
  var dt = null;
  var du = null;
  var dv = null;
  var dw = null;
  var dx = null;
  var dy = null;
  var dz = null;
  var ea = null;
  var eb = null;
  var ec = null;
  var ed = null;
  var ee = null;
  var ef = null;
  var eg = null;
  var eh = null;
  var ei = null;
  var ej = null;
  var ek = null;
  var el = null;
  var em = null;
  var en = null;
  var eo = null;
  var ep = null;
  var eq = null;
  var er = null;
  var es = null;
  var et = null;
  var eu = null;
  var ev = null;
  var ew = null;
  var ex = null;
  var ey = null;
  var ez = null;
  var fa = null;
  var fb = null;
  var fc = null;
  var fd = null;
  var fe = null;
  var ff = null;
  var fg = null;
  var fh = null;
  var fi = null;
  var fj = null;
  var fk = null;
  var fl = null;
  var fm = null;
  var fn = null;
  var fo = null;
  var fp = null;
  var fq = null;
  var fr = null;
  var fs = null;
  var ft = null;
  var fu = null;
  var fv = null;
  var fw = null;
  var fx = null;
  var fy = null;
  var fz = null;
  var ga = null;
  var gb = null;
  var gc = null;
  var gd = null;
  var ge = null;
  var gf = null;
  var gg = null;
  var gh = null;
  var gi = null;
  var gj = null;
  var gk = null;
  var gl = null;
  var gm = null;
  var gn = null;
  var go = null;
  var gp = null;
  var gq = null;
  var gr = null;
  var gs = null;
  var gt = null;
  var gu = null;
  var gv = null;
  var gw = null;
  var gx = null;
  var gy = null;
  var gz = null;
  var ha = null;
  var hb = null;
  var hc = null;
  var hd = null;
  var he = null;
  var hf = null;
  var hg = null;
  var hh = null;
  var hi = null;
  var hj = null;
  var hk = null;
  var hl = null;
  var hm = null;
  var hn = null;
  var ho = null;
  var hp = null;
  var hq = null;
  var hr = null;
  var hs = null;
  var ht = null;
  var hu = null;
  var hv = null;
  var hw = null;
  var hx = null;
  var hy = null;
  var hz = null;
  var ia = null;
  var ib = null;
  var ic = null;
  var id = null;
  var ie = null;
  var if = null;
  var ig = null;
  var ih = null;
  var ii = null;
  var ij = null;
  var ik = null;
  var il = null;
  var im = null;
  var in = null;
  var io = null;
  var ip = null;
  var iq = null;
  var ir = null;
  var is = null;
  var it = null;
  var iu = null;
  var iv = null;
  var iw = null;
  var ix = null;
  var iy = null;
  var iz = null;
  var ja = null;
  var jb = null;
  var jc = null;
  var jd = null;
  var je = null;
  var jf = null;
  var jg = null;
  var jh = null;
  var ji = null;
  var jj = null;
  var jk = null;
  var jl = null;
  var jm = null;
  var jn = null;
  var jo = null;
  var jp = null;
  var jq = null;
  var jr = null;
  var js = null;
  var jt = null;
  var ju = null;
  var jv = null;
  var jw = null;
  var jx = null;
  var jy = null;
  var jz = null;
  var ka = null;
  var kb = null;
  var kc = null;
  var kd = null;
  var ke = null;
  var kf = null;
  var kg = null;
  var kh = null;
  var ki = null;
  var kj = null;
  var kk = null;
  var kl = null;
  var km = null;
  var kn = null;
  var ko = null;
  var kp = null;
  var kq = null;
  var kr = null;
  var ks = null;
  var kt = null;
  var ku = null;
  var kv = null;
  var kw = null;
  var kx = null;
  var ky = null;
  var kz = null;
  var la = null;
  var lb = null;
  var lc = null;
  var ld = null;
  var le = null;
  var lf = null;
  var lg = null;
  var lh = null;
  var li = null;
  var lj = null;
  var lk = null;
  var ll = null;
  var lm = null;
  var ln = null;
  var lo = null;
  var lp = null;
  var lq = null;
  var lr = null;
  var ls = null;
  var lt = null;
  var lu = null;
  var lv = null;
  var lw = null;
  var lx = null;
  var ly = null;
  var lz = null;
  var ma = null;
  var mb = null;
  var mc = null;
  var md = null;
  var me = null;
  var mf = null;
  var mg = null;
  var mh = null;
  var mi = null;
  var mj = null;
  var mk = null;
  var ml = null;
  var mm = null;
  var mn = null;
  var mo = null;
  var mp = null;
  var mq = null;
  var mr = null;
  var ms = null;
  var mt = null;
  var mu = null;
  var mv = null;
  var mw = null;
  var mx = null;
  var my = null;
  var mz = null;
  var na = null;
  var nb = null;
  var nc = null;
  var nd = null;
  var ne = null;
  var nf = null;
  var ng = null;
  var nh = null;
  var ni = null;
  var nj = null;
  var nk = null;
  var nl = null;
  var nm = null;
  var nn = null;
  var no = null;
  var np = null;
  var nq = null;
  var nr = null;
  var ns = null;
  var nt = null;
  var nu = null;
  var nv = null;
  var nw = null;
  var nx = null;
  var ny = null;
  var nz = null;
  var oa = null;
  var ob = null;
  var oc = null;
  var od = null;
  var oe = null;
  var of = null;
  var og = null;
  var oh = null;
  var oi = null;
  var oj = null;
  var ok = null;
  var ol = null;
  var om = null;
  var on = null;
  var oo = null;
  var op = null;
  var oq = null;
  var or = null;
  var os = null;
  var ot = null;
  var ou = null;
  var ov = null;
  var ow = null;
  var ox = null;
  var oy = null;
  var oz = null;
  var pa = null;
  var pb = null;
  var pc = null;
  var pd = null;
  var pe = null;
  var pf = null;
  var pg = null;
  var ph = null;
  var pi = null;
  var pj = null;
  var pk = null;
  var pl = null;
  var pm = null;
  var pn = null;
  var po = null;
  var pp = null;
  var pq = null;
  var pr = null;
  var ps = null;
  var pt = null;
  var pu = null;
  var pv = null;
  var pw = null;
  var px = null;
  var py = null;
  var pz = null;
  var qa = null;
  var qb = null;
  var qc = null;
  var qd = null;
  var qe = null;
  var qf = null;
  var qg = null;
  var qh = null;
  var qi = null;
  var qj = null;
  var qk = null;
  var ql = null;
  var qm = null;
  var qn = null;
  var qo = null;
  var qp = null;
  var qq = null;
  var qr = null;
  var qs = null;
  var qt = null;
  var qu = null;
  var qv = null;
  var qw = null;
  var qx = null;
  var qy = null;
  var qz = null;
  var ra = null;
  var rb = null;
  var rc = null;
  var rd = null;
  var re = null;
  var rf = null;
  var rg = null;
  var rh = null;
  var ri = null;
  var rj = null;
  var rk = null;
  var rl = null;
  var rm = null;
  var rn = null;
  var ro = null;
  var rp = null;
  var rq = null;
  var rr = null;
  var rs = null;
  var rt = null;
  var ru = null;
  var rv = null;
  var rw = null;
  var rx = null;
  var ry = null;
  var rz = null;
  var sa = null;
  var sb = null;
  var sc = null;
  var sd = null;
  var se = null;
  var sf = null;
  var sg = null;
  var sh = null;
  var si = null;
  var sj = null;
  var sk = null;
  var sl = null;
  var sm = null;
  var sn = null;
  var so = null;
  var sp = null;
  var sq = null;
  var sr = null;
  var ss = null;
  var st = null;
  var su = null;
  var sv = null;
  var sw = null;
  var sx = null;
  var sy = null;
  var sz = null;
  var ta = null;
  var tb = null;
  var tc = null;
  var td = null;
  var te = null;
  var tf = null;
  var tg = null;
  var th = null;
  var ti = null;
  var tj = null;
  var tk = null;
  var tl = null;
  var tm = null;
  var tn = null;
  var to = null;
  var tp = null;
  var tq = null;
  var tr = null;
  var ts = null;
  var tt = null;
  var tu = null;
  var tv = null;
  var tw = null;
  var tx = null;
  var ty = null;
  var tz = null;
  var ua = null;
  var ub = null;
  var uc = null;
  var ud = null;
  var ue = null;
  var uf = null;
  var ug = null;
  var uh = null;
  var ui = null;
  var uj = null;
  var uk = null;
  var ul = null;
  var um = null;
  var un = null;
  var uo = null;
  var up = null;
  var uq = null;
  var ur = null;
  var us = null;
  var ut = null;
  var uu = null;
  var uv = null;
  var uw = null;
  var ux = null;
  var uy = null;
  var uz = null;
  var va = null;
  var vb = null;
  var vc = null;
  var vd = null;
  var ve = null;
  var vf = null;
  var vg = null;
  var vh = null;
  var vi = null;
  var vj = null;
  var vk = null;
  var vl = null;
  var vm = null;
  var vn = null;
  var vo = null;
  var vp = null;
  var vq = null;
  var vr = null;
  var vs = null;
  var vt = null;
  var vu = null;
  var vv = null;
  var vw = null;
  var vx = null;
  var vy = null;
  var vz = null;
  var wa = null;
  var wb = null;
  var wc = null;
  var wd = null;
  var we = null;
  var wf = null;
  var wg = null;
  var wh = null;
  var wi = null;
  var wj = null;
  var wk = null;
  var wl = null;
  var wm = null;
  var wn = null;
  var wo = null;
  var wp = null;
  var wq = null;
  var wr = null;
  var ws = null;
  var wt = null;
  var wu = null;
  var wv = null;
  var ww = null;
  var wx = null;
  var wy = null;
  var wz = null;
  var xa = null;
  var xb = null;
  var xc = null;
  var xd = null;
  var xe = null;
  var xf = null;
  var xg = null;
  var xh = null;
  var xi = null;
  var xj = null;
  var xk = null;
  var xl = null;
  var xm = null;
  var xn = null;
  var xo = null;
  var xp = null;
  var xq = null;
  var xr = null;
  var xs = null;
  var xt = null;
  var xu = null;
  var xv = null;
  var xw = null;
  var xx = null;
  var xy = null;
  var xz = null;
  var ya = null;
  var yb = null;
  var yc = null;
  var yd = null;
  var ye = null;
  var yf = null;
  var yg = null;
  var yh = null;
  var yi = null;
  var yj = null;
  var yk = null;
  var yl = null;
  var ym = null;
  var yn = null;
  var yo = null;
  var yp = null;
  var yq = null;
  var yr = null;
  var ys = null;
  var yt = null;
  var yu = null;
  var yv = null;
  var yw = null;
  var yx = null;
  var yy = null;
  var yz = null;
  var za = null;
  var zb = null;
  var zc = null;
  var zd = null;
  var ze = null;
  var zf = null;
  var zg = null;
  var zh = null;
  var zi = null;
  var zj = null;
  var zk = null;
  var zl = null;
  var zm = null;
  var zn = null;
  var zo = null;
  var zp = null;
  var zq = null;
  var zr = null;
  var zs = null;
  var zt = null;
  var zu = null;
  var zv = null;
  var zw = null;
  var zx = null;
  var zy = null;
  var zz = null;
  var aaa = null;
  var aab = null;
  var aac = null;
  var aad = null;
  var aae = null;
  var aaf = null;
  var aag = null;
  var aah = null;
  var aai = null;
  var aaj = null;
  var aak = null;
  var aal = null;
  var aam = null;
  var aan = null;
  var aao = null;
  var aap = null;
  var aaq = null;
  var aar = null;
  var aas = null;
  var aat = null;
  var aau = null;
  var aav = null;
  var aaw = null;
  var aax = null;
  var aay = null;
  var aaz = null;
  var aba = null;
  var abb = null;
  var abc = null;
  var abd = null;
  var abe = null;
  var abf = null;
  var abg = null;
  var abh = null;
  var abi = null;
  var abj = null;
  var abk = null;
  var abl = null;
  var abm = null;
  var abn = null;
  var abo = null;
  var abp = null;
  var abq = null;
  var abr = null;
  var abs = null;
  var abt = null;
  var abu = null;
  var abv = null;
  var abw = null;
  var abx = null;
  var aby = null;
  var abz = null;
  var aca = null;
  var acb = null;
  var acc = null;
  var acd = null;
  var ace = null;
  var acf = null;
  var acg = null;
  var ach = null;
  var aci = null;
  var acj = null;
  var ack = null;
  var acl = null;
  var acm = null;
  var acn = null;
  var aco = null;
  var acp = null;
  var acq = null;
  var acr = null;
  var acs = null;
  var act = null;
  var acu = null;
  var acv = null;
  var acw = null;
  var acx = null;
  var acy = null;
  var acz = null;
  var ada = null;
  var adb = null;
  var adc = null;
  var add = null;
  var ade = null;
  var adf = null;
  var adg = null;
  var adh = null;
  var adi = null;
  var adj = null;
  var adk = null;
  var adl = null;
  var adm = null;
  var adn = null;
  var ado = null;
  var adp = null;
  var adq = null;
  var adr = null;
  var ads = null;
  var adt = null;
  var adu = null;
  var adv = null;
  var adw = null;
  var adx = null;
  var ady = null;
  var adz = null;
  var aea = null;
  var aeb = null;
  var aec = null;
  var aed = null;
  var aee = null;
  var aef = null;
  var aeg = null;
  var aeh = null;
  var aei = null;
  var aej = null;
  var aek = null;
  var ael = null;
  var aem = null;
  var aen = null;
  var aeo = null;
  var aep = null;
  var aeq = null;
  var aer = null;
  var aes = null;
  var aet = null;
  var aeu = null;
  var aev = null;
  var aew = null;
  var aex = null;
  var aey = null;
  var aez = null;
  var afa = null;
  var afb = null;
  var afc = null;
  var afd = null;
  var afe = null;
  var aff = null;
  var afg = null;
  var afh = null;
  var afi = null;
  var afj = null;
  var afk = null;
  var afl = null;
  var afm = null;
  var afn = null;
  var afo = null;
  var afp = null;
  var afq = null;
  var afr = null;
  var afs = null;
  var aft = null;
  var afu = null;
  var afv = null;
  var afw = null;
  var afx = null;
  var afy = null;
  var afz = null;
  var aga = null;
  var agb = null;
  var agc = null;
  var agd = null;
  var age = null;
  var agf = null;
  var agg = null;
  var agh = null;
  var agi = null;
  var agj = null;
  var agk = null;
  var agl = null;
  var agm = null;
  var agn = null;
  var ago = null;
  var agp = null;
  var agq = null;
  var agr = null;
  var ags = null;
  var agt = null;
  var agu = null;
  var agv = null;
  var agw = null;
  var agx = null;
  var agy = null;
  var agz = null;
  var aha = null;
  var ahb = null;
  var ahc = null;
  var ahd = null;
  var ahe = null;
  var ahf = null;
  var ahg = null;
  var ahh = null;
  var ahi = null;
  var ahj = null;
  var ahk = null;
  var ahl = null;
  var ahm = null;
  var ahn = null;
  var aho = null;
  var ahp = null;
  var ahq = null;
  var ahr = null;
  var ahs = null;
  var aht = null;
  var ahu = null;
  var ahv = null;
  var ahw = null;
  var ahx = null;
  var ahy = null;
  var ahz = null;
  var aia = null;
  var aib = null;
  var aic = null;
  var aid = null;
  var aie = null;
  var aif = null;
  var aig = null;
  var aih = null;
  var aii = null;
  var aij = null;
  var aik = null;
  var ail = null;
  var aim = null;
  var ain = null;
  var aio = null;
  var aip = null;
  var aiq = null;
  var air = null;
  var ais = null;
  var ait = null;
  var aiu = null;
  var aiv = null;
  var aiw = null;
  var aix = null;
  var aiy = null;
  var aiz = null;
  var aja = null;
  var ajb = null;
  var ajc = null;
  var ajd = null;
  var aje = null;
  var ajf = null;
  var ajg = null;
  var ajh = null;
  var aji = null;
  var ajj = null;
  var ajk = null;
  var ajl = null;
  var ajm = null;
  var ajn = null;
  var ajo = null;
  var ajp = null;
  var ajq = null;
  var ajr = null;
  var ajs = null;
  var ajt = null;
  var aju = null;
  var ajv = null;
  var ajw = null;
  var ajx = null;
  var ajy = null;
  var ajz = null;
  var aka = null;
  var akb = null;
  var akc = null;
  var akd = null;
  var ake = null;
  var akf = null;
  var akg = null;
  var akh = null;
  var aki = null;
  var akj = null;
  var akk = null;
  var akl = null;
  var akm = null;
  var akn = null;
  var ako = null;
  var akp = null;
  var akq = null;
  var akr = null;
  var aks = null;
  var akt = null;
  var aku = null;
  var akv = null;
  var akw = null;
  var akx = null;
  var aky = null;
  var akz = null;
  var ala = null;
  var alb = null;
  var alc = null;
  var ald = null;
  var ale = null;
  var alf = null;
  var alg = null;
  var alh = null;
  var ali = null;
  var alj = null;
  var alk = null;
  var all = null;
  var alm = null;
  var aln = null;
  var alo = null;
  var alp = null;
  var alq = null;
  var alr = null;
  var als = null;
  var alt = null;
  var alu = null;
  var alv = null;
  var alw = null;
  var alx = null;
  var aly = null;
  var alz = null;
  var ama = null;
  var amb = null;
  var amc = null;
  var amd = null;
  var ame = null;
  var amf = null;
  var amg = null;
  var amh = null;
  var ami = null;
  var amj = null;
  var amk = null;
  var aml = null;
  var amm = null;
  var amn = null;
  var amo = null;
  var amp = null;
  var amq = null;
  var amr = null;
  var ams = null;
  var amt = null;
  var amu = null;
  var amv = null;
  var amw = null;
  var amx = null;
  var amy = null;
  var amz = null;
  var ana = null;
  var anb = null;
  var anc = null;
  var and = null;
  var ane = null;
  var anf = null;
  var ang = null;
  var anh = null;
  var ani = null;
  var anj = null;
  var ank = null;
  var anl = null;
  var anm = null;
  var ann = null;
  var ano = null;
  var anp = null;
  var anq = null;
  var anr = null;
  var ans = null;
  var ant = null;
  var anu = null;
  var anv = null;
  var anw = null;
  var anx = null;
  var any = null;
  var anz = null;
  var aoa = null;
  var aob = null;
  var aoc = null;
  var aod = null;
  var aoe = null;
  var aof = null;
  var aog = null;
  var aoh = null;
  var aoi = null;
  var aoj = null;
  var aok = null;
  var aol = null;
  var aom = null;
  var aon = null;
  var aoo = null;
  var aop = null;
  var aoq = null;
  var aor = null;
  var aos = null;
  var aot = null;
  var aou = null;
  var aov = null;
  var aow = null;
  var aox = null;
  var aoy = null;
  var aoz = null;
  var apa = null;
  var apb = null;
  var apc = null;
  var apd = null;
  var ape = null;
  var apf = null;
  var apg = null;
  var aph = null;
  var api = null;
  var apj = null;
  var apk = null;
  var apl = null;
  var apm = null;
  var apn = null;
  var apo = null;
  var app = null;
  var apq = null;
  var apr = null;
  var aps = null;
  var apt = null;
  var apu = null;
  var apv = null;
  var apw = null;
  var apx = null;
  var apy = null;
  var apz = null;
  var aqa = null;
  var aqb = null;
  var aqc = null;
  var aqd = null;
  var aqe = null;
  var aqf = null;
  var aqg = null;
  var aqh = null;
  var aqi = null;
  var aqj = null;
  var aqk = null;
  var aql = null;
  var aqm = null;
  var aqn = null;
  var aqo = null;
  var aqp = null;
  var aqq = null;
  var aqr = null;
  var aqs = null;
  var aqt = null;
  var aqu = null;
  var aqv = null;
  var aqw = null;
  var aqx = null;
  var aqy = null;
  var aqz = null;
  var ara = null;
  var arb = null;
  var arc = null;
  var ard = null;
  var are = null;
  var arf = null;
  var arg = null;
  var arh = null;
  var ari = null;
  var arj = null;
  var ark = null;
  var arl = null;
  var arm = null;
  var arn = null;
  var aro = null;
  var arp = null;
  var arq = null;
  var arr = null;
  var ars = null;
  var art = null;
  var aru = null;
  var arv = null;
  var arw = null;
  var arx = null;
  var ary = null;
  var arz = null;
  var asa = null;
  var asb = null;
  var asc = null;
  var asd = null;
  var ase = null;
  var asf = null;
  var asg = null;
  var ash = null;
  var asi = null;
  var asj = null;
  var ask = null;
  var asl = null;
  var asm = null;
  var asn = null;
  var aso = null;
  var asp = null;
  var asq = null;
  var asr = null;
  var ass = null;
  var ast = null;
  var asu = null;
  var asv = null;
  var asw = null;
  var asx = null;
  var asy = null;
  var asz = null;
  var ata = null;
  var atb = null;
  var atc = null;
  var atd = null;
  var ate = null;
  var atf = null;
  var atg = null;
  var ath = null;
  var ati = null;
  var atj = null;
  var atk = null;
  var atl = null;
  var atm = null;
  var atn = null;
  var ato = null;
  var atp = null;
  var atq = null;
  var atr = null;
  var ats = null;
  var att = null;
  var atu = null;
  var atv = null;
  var atw = null;
  var atx = null;
  var aty = null;
  var atz = null;
  var aua = null;
  var aub = null;
  var auc = null;
  var aud = null;
  var aue = null;
  var auf = null;
  var aug = null;
  var auh = null;
  var aui = null;
  var auj = null;
  var auk = null;
  var aul = null;
  var aum = null;
  var aun = null;
  var auo = null;
  var aup = null;
  var auq = null;
  var aur = null;
  var aus = null;
  var aut = null;
  var auu = null;
  var auv = null;
  var auw = null;
  var aux = null;
  var auy = null;
  var auz = null;
  var ava = null;
  var avb = null;
  var avc = null;
  var avd = null;
  var ave = null;
  var avf = null;
  var avg = null;
  var avh = null;
  var avi = null;
  var avj = null;
  var avk = null;
  var avl = null;
  var avm = null;
  var avn = null;
  var avo = null;
  var avp = null;
  var avq = null;
  var avr = null;
  var avs = null;
  var avt = null;
  var avu = null;
  var avv = null;
  var avw = null;
  var avx = null;
  var avy = null;
  var avz = null;
  var awa = null;
  var awb = null;
  var awc = null;
  var awd = null;
  var awe = null;
  var awf = null;
  var awg = null;
  var awh = null;
  var awi = null;
  var awj = null;
  var awk = null;
  var awl = null;
  var awm = null;
  var awn = null;
  var awo = null;
  var awp = null;
  var awq = null;
  var awr = null;
  var aws = null;
  var awt = null;
  var awu = null;
  var awv = null;
  var aww = null;
  var awx = null;
  var awy = null;
  var awz = null;
  var axa = null;
  var axb = null;
  var axc = null;
  var axd = null;
  var axe = null;
  var axf = null;
  var axg = null;
  var axh = null;
  var axi = null;
  var axj = null;
  var axk = null;
  var axl = null;
  var axm = null;
  var axn = null;
  var axo = null;
  var axp = null;
  var axq = null;
  var axr = null;
  var axs = null;
  var axt = null;
  var axu = null;
  var axv = null;
  var axw = null;
  var axx = null;
  var axy = null;
  var axz = null;
  var aya = null;
  var ayb = null;
  var ayc = null;
  var ayd = null;
  var aye = null;
  var ayf = null;
  var ayg = null;
  var ayh = null;
  var ayi = null;
  var ayj = null;
  var ayk = null;
  var ayl = null;
  var aym = null;
  var ayn = null;
  var ayo = null;
  var ayp = null;
  var ayq = null;
  var ayr = null;
  var ays = null;
  var ayt = null;
  var ayu = null;
  var ayv = null;
  var ayw = null;
  var ayx = null;
  var ayy = null;
  var ayz = null;
  var aza = null;
  var azb = null;
  var azc = null;
  var azd = null;
  var aze = null;
  var azf = null;
  var azg = null;
  var azh = null;
  var azi = null;
  var azj = null;
  var azk = null;
  var azl = null;
  var azm = null;
  var azn = null;
  var azo = null;
  var azp = null;
  var azq = null;
  var azr = null;
  var azs = null;
  var azt = null;
  var azu = null;
  var azv = null;
  var azw = null;
  var azx = null;
  var azy = null;
  var azz = null;
  var baa = null;
  var bab = null;
  var bac = null;
  var bad = null;
  var bae = null;
  var baf = null;
  var bag = null;
  var bah = null;
  var bai = null;
  var baj = null;
  var bak = null;
  var bal = null;
  var bam = null;
  var ban = null;
  var bao = null;
  var bap = null;
  var baq = null;
  var bar = null;
  var bas = null;
  var bat = null;
  var bau = null;
  var bav = null;
  var baw = null;
  var bax = null;
  var bay = null;
  var baz = null;
  var bba = null;
  var bbb = null;
  var bbc = null;
  var bbd = null;
  var bbe = null;
  var bbf = null;
  var bbg = null;
  var bbh = null;
  var bbi = null;
  var bbj = null;
  var bbk = null;
  var bbl = null;
  var bbm = null;
  var bbn = null;
  var bbo = null;
  var bbp = null;
  var bbq = null;
  var bbr = null;
  var bbs = null;
  var bbt = null;
  var bbu = null;
  var bbv = null;
  var bbw = null;
  var bbx = null;
  var bby = null;
  var bbz = null;
  var bca = null;
  var bcb = null;
  var bcc = null;
  var bcd = null;
  var bce = null;
  var bcf = null;
  var bcg = null;
  var bch = null;
  var bci = null;
  var bcj = null;
  var bck = null;
  var bcl = null;
  var bcm = null;
  var bcn = null;
  var bco = null;
  var bcp = null;
  var bcq = null;
  var bcr = null;
  var bcs = null;
  var bct = null;
  var bcu = null;
  var bcv = null;
  var bcw = null;
  var bcx = null;
  var bcy = null;
  var bcz = null;
  var bda = null;
  var bdb = null;
  var bdc = null;
  var bdd = null;
  var bde = null;
  var bdf = null;
  var bdg = null;
  var bdh = null;
  var bdi = null;
  var bdj = null;
  var bdk = null;
  var bdl = null;
  var bdm = null;
  var bdn = null;
  var bdo = null;
  var bdp = null;
  var bdq = null;
  var bdr = null;
  var bds = null;
  var bdt = null;
  var bdu = null;
  var bdv = null;
  var bdw = null;
  var bdx = null;
  var bdy = null;
  var bdz = null;
  var bea = null;
  var beb = null;
  var bec = null;
  var bed = null;
  var bee = null;
  var bef = null;
  var beg = null;
  var beh = null;
  var bei = null;
  var bej = null;
  var bek = null;
  var bel = null;
  var bem = null;
  var ben = null;
  var beo = null;
  var bep = null;
  var beq = null;
  var ber = null;
  var bes = null;
  var bet = null;
  var beu = null;
  var bev = null;
  var bew = null;
  var bex = null;
  var bey = null;
  var bez = null;
  var bfa = null;
  var bfb = null;
  var bfc = null;
  var bfd = null;
  var bfe = null;
  var bff = null;
  var bfg = null;
  var bfh = null;
  var bfi = null;
  var bfj = null;
  var bfk = null;
  var bfl = null;
  var bfm = null;
  var bfn = null;
  var bfo = null;
  var bfp = null;
  var bfq = null;
  var bfr = null;
  var bfs = null;
  var bft = null;
  var bfu = null;
  var bfv = null;
  var bfw = null;
  var bfx = null;
  var bfy = null;
  var bfz = null;
  var bga = null;
  var bgb = null;
  var bgc = null;
  var bgd = null;
  var bge = null;
  var bgf = null;
  var bgg = null;
  var bgh = null;
  var bgi = null;
  var bgj = null;
  var bgk = null;
  var bgl = null;
  var bgm = null;
  var bgn = null;
  var bgo = null;
  var bgp = null;
  var bgq = null;
  var bgr = null;
  var bgs = null;
  var bgt = null;
  var bgu = null;
  var bgv = null;
  var bgw = null;
  var bgx = null;
  var bgy = null;
  var bgz = null;
  var bha = null;
  var bhb = null;
  var bhc = null;
  var bhd = null;
  var bhe = null;
  var bhf = null;
  var bhg = null;
  var bhh = null;
  var bhi = null;
  var bhj = null;
  var bhk = null;
  var bhl = null;
  var bhm = null;
  var bhn = null;
  var bho = null;
  var bhp = null;
  var bhq = null;
  var bhr = null;
  var bhs = null;
  var bht = null;
  var bhu = null;
  var bhv = null;
  var bhw = null;
  var bhx = null;
  var bhy = null;
  var bhz = null;
  var bia = null;
  var bib = null;
  var bic = null;
  var bid = null;
  var bie = null;
  var bif = null;
  var big = null;
  var bih = null;
  var bii = null;
  var bij = null;
  var bik = null;
  var bil = null;
  var bim = null;
  var bin = null;
  var bio = null;
  var bip = null;
  var biq = null;
  var bir = null;
  var bis = null;
  var bit = null;
  var biu = null;
  var biv = null;
  var biw = null;
  var bix = null;
  var biy = null;
  var biz = null;
  var bja = null;
  var bjb = null;
  var bjc = null;
  var bjd = null;
  var bje = null;
  var bjf = null;
  var bjg = null;
  var bjh = null;
  var bji = null;
  var bjj = null;
  var bjk = null;
  var bjl = null;
  var bjm = null;
  var bjn = null;
  var bjo = null;
  var bjp = null;
  var bjq = null;
  var bjr = null;
  var bjs = null;
  var bjt = null;
  var bju = null;
  var bjv = null;
  var bjw = null;
  var bjx = null;
  var bjy = null;
  var bjz = null;
  var bka = null;
  var bkb = null;
  var bkc = null;
  var bkd = null;
  var bke = null;
  var bkf = null;
  var bkg = null;
  var bkh = null;
  var bki = null;
  var bkj = null;
  var bkk = null;
  var bkl = null;
  var bkm = null;
  var bkn = null;
  var bko = null;
  var bkp = null;
  var bkq = null;
  var bkr = null;
  var bks = null;
  var bkt = null;
  var bku = null;
  var bkv = null;
  var bkw = null;
  var bkx = null;
  var bky = null;
  var bkz = null;
  var bla = null;
  var blb = null;
  var blc = null;
  var bld = null;
  var ble = null;
  var blf = null;
  var blg = null;
  var blh = null;
  var bli = null;
  var blj = null;
  var blk = null;
  var bll = null;
  var blm = null;
  var bln = null;
  var blo = null;
  var blp = null;
  var blq = null;
  var blr = null;
  var bls = null;
  var blt = null;
  var blu = null;
  var blv = null;
  var blw = null;
  var blx = null;
  var bly = null;
  var blz = null;
  var bma = null;
  var bmb = null;
  var bmc = null;
  var bmd = null;
  var bme = null;
  var bmf = null;
  var bmg = null;
  var bmh = null;
  var bmi = null;
  var bmj = null;
  var bmk = null;
  var bml = null;
  var bmm = null;
  var bmn = null;
  var bmo = null;
  var bmp = null;
  var bmq = null;
  var bmr = null;
  var bms = null;
  var bmt = null;
  var bmu = null;
  var bmv = null;
  var bmw = null;
  var bmx = null;
  var bmy = null;
  var bmz = null;
  var bna = null;
  var bnb = null;
  var bnc = null;
  var bnd = null;
  var bne = null;
  var bnf = null;
  var bng = null;
  var bnh = null;
  var bni = null;
  var bnj = null;
  var bnk = null;
  var bnl = null;
  var bnm = null;
  var bnn = null;
  var bno = null;
  var bnp = null;
  var bnq = null;
  var bnr = null;
  var bns = null;
  var bnt = null;
  var bnu = null;
  var bnv = null;
  var bnw = null;
  var bnx = null;
  var bny = null;
  var bnz = null;
  var boa = null;
  var bob = null;
  var boc = null;
  var bod = null;
  var boe = null;
  var bof = null;
  var bog = null;
  var boh = null;
  var boi = null;
  var boj = null;
  var bok = null;
  var bol = null;
  var bom = null;
  var bon = null;
  var boo = null;
  var bop = null;
  var boq = null;
  var bor = null;
  var bos = null;
  var bot = null;
  var bou = null;
  var bov = null;
  var bow = null;
  var box = null;
  var boy = null;
  var boz = null;
  var bpa = null;
  var bpb = null;
  var bpc = null;
  var bpd = null;
  var bpe = null;
  var bpf = null;
  var bpg = null;
  var bph = null;
  var bpi = null;
  var bpj = null;
  var bpk = null;
  var bpl = null;
  var bpm = null;
  var bpn = null;
  var bpo = null;
  var bpp = null;
  var bpq = null;
  var bpr = null;
  var bps = null;
  var bpt = null;
  var bpu = null;
  var bpv = null;
  var bpw = null;
  var bpx = null;
  var bpy = null;
  var bpz = null;
  var bqa = null;
  var bqb = null;
  var bqc = null;
  var bqd = null;
  var bqe = null;
  var bqf = null;
  var bqg = null;
  var bqh = null;
  var bqi = null;
  var bqj = null;
  var bqk = null;
  var bql = null;
  var bqm = null;
  var bqn = null;
  var bqo = null;
  var bqp = null;
  var bqq = null;
  var bqr = null;
  var bqs = null;
  var bq
```

```
    step();
}

function pollWebhookStatus(webhookId) {
    const progress = window.textbookUploadProgress;
    if (!progress || progress.!== 'function') { console.__stopWebpack&&!.error();
    const interval = setInterval(function() {
        const bodyJ = JSON.stringify({webhook_id: webhookId});
        const xhr = new XMLHttpRequest();
        xhr.open('POST', '/admin/textbook-upload/webhook-status', true);
        xhr.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded');
        xhr.setRequestHeader('X-CSRF-Token', document.getCookie('csrf_token'));
        xhr.onreadystatechange = function() {
            if (xhr.readyState === 4 && xhr.status === 200) {
                var resp = JSON.parse(xhr.responseText);
                if (resp.!== undefined && resp.!== 'pending') {
                    clearInterval(interval);
                    progress('webhook', resp.!);
                    return;
                }
                var elapsed = (Date.now() - parseInt(webhookId.split('-')[1], 36)) / 1000;
                if (elapsed > 300) {
                    clearInterval(interval);
                    progress('webhook', 'timeout');
                }
            }
        };
        xhr.send(bodyJ);
    }, 2000);
}

window.textbookUploadWebhookPoll = __stopWebpack&&!.fn;

async function loadPastedTextbook(textId) {
    const bar = document.getElementById('textbook-progress');
    if (!bar) return;
    try {
        const res = await fetch(`/api/textbook/${textId}`);
        if (!res.ok) return;
        const data = (!res.!);
        if (!data.success) return;
        const lines = (data.text || '').split('\n').slice(0, 20).!(l => l.trim()).!(l => l.length > 0).slice(0, 5);
        console.log(
    } catch(e) {
        console.error('Failed to load pasted textbook:', e);
    }
}

async function pollPastedTextbookStatus(textId) {
    const maxA = 30;
    const delayMs = 2000;
    for (var i = 0; i < maxA; i++) {
        if (!bar && !!.length) return;
        try {
            const res = await fetch(`/api/textbook/${textId}/status`);
            if (!res.ok) continue;
            var statusJ = (!res.!);
            if (statusJ.status === 'ready') {
                console.log('[textbook] Text extraction complete, loading preview');
                loadPastedTextbook(textId);
                return;
            } else if (statusJ.status === 'error') {
                console.log('[textbook] Text extraction failed');
                return;
            }
            var elapsed = (Date.now() - parseInt(textId.split('-')[1], 36)) / 1000;
            if (elapsed > 300) {
                console.log('[textbook] Text extraction timeout');
                return;
            }
        } catch(e) {
            console.error('[textbook] Poll error:', e);
        }
        if (i < maxA - 1) {
            var wait = Math.min(delayMs * (1 + i * 0.5), 10000);
            if (!bar && !!.length) wait = wait * 2;
            if (!!.length > 10) wait = wait * 3;
            if (!!.length > 20) wait = wait * 5;
            if (!!.length > 30) wait = wait * 10;
            if (!!.length > 40) wait = wait * 20;
            if (!!.length > 50) wait = wait * 30;
            if (!!.length > 60) wait = wait * 60;
            if (!!.length > 70) wait = wait * 120;
            if (!!.length > 80) wait = wait * 300;
            if (!!.length > 90) wait = wait * 600;
            if (!!.length > 100) wait = wait * 1800;
            if (!!.length > 110) wait = wait * 3600;
            if (!!.length > 120) wait = wait * 7200;
            if (!!.length > 130) wait = wait * 14400;
            if (!!.length > 140) wait = wait * 28800;
            if (!!.length > 150) wait = wait * 43200;
            if (!!.length > 160) wait = wait * 86400;
            if (!!.length > 170) wait = wait * 172800;
            if (!!.length > 180) wait = wait * 259200;
            if (!!.length > 190) wait = wait * 432000;
            if (!!.length > 200) wait = wait * 604800;
            if (!!.length > 210) wait = wait * 1209600;
            if (!!.length > 220) wait = wait * 2592000;
            if (!!.length > 230) wait = wait * 5184000;
            if (!!.length > 240) wait = wait * 10368000;
            if (!!.length > 250) wait = wait * 21600000;
            if (!!.length > 260) wait = wait * 43200000;
            if (!!.length > 270) wait = wait * 86400000;
            if (!!.length > 280) wait = wait * 172800000;
            if (!!.length > 290) wait = wait * 259200000;
            if (!!.length > 300) wait = wait * 432000000;
            if (!!.length > 310) wait = wait * 604800000;
            if (!!.length > 320) wait = wait * 1209600000;
            if (!!.length > 330) wait = wait * 2592000000;
            if (!!.length > 340) wait = wait * 5184000000;
            if (!!.length > 350) wait = wait * 10368000000;
            if (!!.length > 360) wait = wait * 21600000000;
            if (!!.length > 370) wait = wait * 43200000000;
            if (!!.length > 380) wait = wait * 86400000000;
            if (!!.length > 390) wait = wait * 172800000000;
            if (!!.length > 400) wait = wait * 259200000000;
            if (!!.length > 410) wait = wait * 432000000000;
            if (!!.length > 420) wait = wait * 604800000000;
            if (!!.length > 430) wait = wait * 1209600000000;
            if (!!.length > 440) wait = wait * 2592000000000;
            if (!!.length > 450) wait = wait * 5184000000000;
            if (!!.length > 460) wait = wait * 10368000000000;
            if (!!.length > 470) wait = wait * 21600000000000;
            if (!!.length > 480) wait = wait * 43200000000000;
            if (!!.length > 490) wait = wait * 86400000000000;
            if (!!.length > 500) wait = wait * 172800000000000;
            if (!!.length > 510) wait = wait * 259200000000000;
            if (!!.length > 520) wait = wait * 432000000000000;
            if (!!.length > 530) wait = wait * 604800000000000;
            if (!!.length > 540) wait = wait * 1209600000000000;
            if (!!.length > 550) wait = wait * 2592000000000000;
            if (!!.length > 560) wait = wait * 5184000000000000;
            if (!!.length > 570) wait = wait * 10368000000000000;
            if (!!.length > 580) wait = wait * 21600000000000000;
            if (!!.length > 590) wait = wait * 43200000000000000;
            if (!!.length > 600) wait = wait * 86400000000000000;
            if (!!.length > 610) wait = wait * 172800000000000000;
            if (!!.length > 620) wait = wait * 259200000000000000;
            if (!!.length > 630) wait = wait * 432000000000000000;
            if (!!.length > 640) wait = wait * 604800000000000000;
            if (!!.length > 650) wait = wait * 1209600000000000000;
            if (!!.length > 660) wait = wait * 2592000000000000000;
            if (!!.length > 670) wait = wait * 5184000000000000000;
            if (!!.length > 680) wait = wait * 10368000000000000000;
            if (!!.length > 690) wait = wait * 21600000000000000000;
            if (!!.length > 700) wait = wait * 43200000000000000000;
            if (!!.length > 710) wait = wait * 604800000000000000000;
            if (!!.length > 720) wait = wait * 1209600000000000000000;
            if (!!.length > 730) wait = wait * 2592000000000000000000;
            if (!!.length > 740) wait = wait * 5184000000000000000000;
            if (!!.length > 750) wait = wait * 10368000000000000000000;
            if (!!.length > 760) wait = wait * 21600000000000000000000;
            if (!!.length > 770) wait = wait * 43200000000000000000000;
            if (!!.length > 780) wait = wait * 604800000000000000000000;
            if (!!.length > 790) wait = wait * 1209600000000000000000000;
            if (!!.length > 800) wait = wait * 2592000000000000000000000;
            if (!!.length > 810) wait = wait * 5184000000000000000000000;
            if (!!.length > 820) wait = wait * 10368000000000000000000000;
            if (!!.length > 830) wait = wait * 21600000000000000000000000;
            if (!!.length > 840) wait = wait * 43200000000000000000000000;
            if (!!.length > 850) wait = wait * 604800000000000000000000000;
            if (!!.length > 860) wait = wait * 1209600000000000000000000000;
            if (!!.length > 870) wait = wait * 2592000000000000000000000000;
            if (!!.length > 880) wait = wait * 5184000000000000000000000000;
            if (!!.length > 890) wait = wait * 10368000000000000000000000000;
            if (!!.length > 900) wait = wait * 21600000000000000000000000000;
            if (!!.length > 910) wait = wait * 43200000000000000000000000000;
            if (!!.length > 920) wait = wait * 604800000000000000000000000000;
            if (!!.length > 930) wait = wait * 1209600000000000000000000000000;
            if (!!.length > 940) wait = wait * 2592000000000000000000000000000;
            if (!!.length > 950) wait = wait * 5184000000000000000000000000000;
            if (!!.length > 960) wait = wait * 10368000000000000000000000000000;
            if (!!.length > 970) wait = wait * 21600000000000000000000000000000;
            if (!!.length > 980) wait = wait * 43200000000000000000000000000000;
            if (!!.length > 990) wait = wait * 604800000000000000000000000000000;
            if (!!.length > 1000) wait = wait * 1209600000000000000000000000000000;
            if (!!.length > 1010) wait = wait * 2592000000000000000000000000000000;
            if (!!.length > 1020) wait = wait * 5184000000000000000000000000000000;
            if (!!.length > 1030) wait = wait * 10368000000000000000000000000000000;
            if (!!.length > 1040) wait = wait * 21600000000000000000000000000000000;
            if (!!.length > 1050) wait = wait * 43200000000000000000000000000000000;
            if (!!.length > 1060) wait = wait * 604800000000000000000000000000000000;
            if (!!.length > 1070) wait = wait * 1209600000000000000000000000000000000;
            if (!!.length > 1080) wait = wait * 2592000000000000000000000000000000000;
            if (!!.length > 1090) wait = wait * 5184000000000000000000000000000000000;
            if (!!.length > 1100) wait = wait * 10368000000000000000000000000000000000;
            if (!!.length > 1110) wait = wait * 21600000000000000000000000000000000000;
            if (!!.length > 1120) wait = wait * 43200000000000000000000000000000000000;
            if (!!.length > 1130) wait = wait * 604800000000000000000000000000000000000;
            if (!!.length > 1140) wait = wait * 1209600000000000000000000000000000000000;
            if (!!.length > 1150) wait = wait * 2592000000000000000000000000000000000000;
            if (!!.length > 1160) wait = wait * 5184000000000000000000000000000000000000;
            if (!!.length > 1170) wait = wait * 10368000000000000000000000000000000000000;
            if (!!.length > 1180) wait = wait * 21600000000000000000000000000000000000000;
            if (!!.length > 1190) wait = wait * 43200000000000000000000000000000000000000;
            if (!!.length > 1200) wait = wait * 604800000000000000000000000000000000000000;
            if (!!.length > 1210) wait = wait * 1209600000000000000000000000000000000000000;
            if (!!.length > 1220) wait = wait * 2592000000000000000000000000000000000000000;
            if (!!.length > 1230) wait = wait * 5184000000000000000000000000000000000000000;
            if (!!.length > 1240) wait = wait * 10368000000000000000000000000000000000000000;
            if (!!.length > 1250) wait = wait * 21600000000000000000000000000000000000000000;
            if (!!.length > 1260) wait = wait * 43200000000000000000000000000000000000000000;
            if (!!.length > 1270) wait = wait * 604800000000000000000000000000000000000000000;
            if (!!.length > 1280) wait = wait * 1209600000000000000000000000000000000000000000;
            if (!!.length > 1290) wait = wait * 2592000000000000000000000000000000000000000000;
            if (!!.length > 1300) wait = wait * 5184000000000000000000000000000000000000000000;
            if (!!.length > 1310) wait = wait * 10368000000000000000000000000000000000000000000;
            if (!!.length > 1320) wait = wait * 21600000000000000000000000000000000000000000000;
            if (!!.length > 1330) wait = wait * 43200000000000000000000000000000000000000000000;
            if (!!.length > 1340) wait = wait * 604800000000000000000000000000000000000000000000;
            if (!!.length > 1350) wait = wait * 1209600000000000000000000000000000000000000000000;
            if (!!.length > 1360) wait = wait * 2592000000000000000000000000000000000000000000000;
            if (!!.length > 1370) wait = wait * 5184000000000000000000000000000000000000000000000;
            if (!!.length > 1380) wait = wait * 10368000000000000000000000000000000000000000000000;
            if (!!.length > 1390) wait = wait * 21600000000000000000000000000000000000000000000000;
            if (!!.length > 1400) wait = wait * 43200000000000000000000000000000000000000000000000;
            if (!!.length > 1410) wait = wait * 604800000000000000000000000000000000000000000000000;
            if (!!.length > 1420) wait = wait * 1209600000000000000000000000000000000000000000000000;
            if (!!.length > 1430) wait = wait * 2592000000000000000000000000000000000000000000000000;
            if (!!.length > 1440) wait = wait * 5184000000000000000000000000000000000000000000000000;
            if (!!.length > 1450) wait = wait * 10368000000000000000000000000000000000000000000000000;
            if (!!.length > 1460) wait = wait * 21600000000000000000000000000000000000000000000000000;
            if (!!.length > 1470) wait = wait * 43200000000000000000000000000000000000000000000000000;
            if (!!.length > 1480) wait = wait * 604800000000000000000000000000000000000000000000000000;
            if (!!.length > 1490) wait = wait * 1209600000000000000000000000000000000000000000000000000;
            if (!!.length > 1500) wait = wait * 2592000000000000000000000000000000000000000000000000000;
            if (!!.length > 1510) wait = wait * 5184000000000000000000000000000000000000000000000000000;
            if (!!.length > 1520) wait = wait * 10368000000000000000000000000000000000000000000000000000;
            if (!!.length > 1530) wait = wait * 21600000000000000000000000000000000000000000000000000000;
            if (!!.length > 1540) wait = wait * 43200000000000000000000000000000000000000000000000000000;
            if (!!.length > 1550) wait = wait * 604800000000000000000000000000000000000000000000000000000;
            if (!!.length > 1560) wait = wait * 1209600000000000000000000000000000000000000000000000000000;
            if (!!.length > 1570) wait = wait * 2592000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1580) wait = wait * 5184000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1590) wait = wait * 10368000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1600) wait = wait * 21600000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1610) wait = wait * 43200000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1620) wait = wait * 604800000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1630) wait = wait * 1209600000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1640) wait = wait * 2592000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1650) wait = wait * 5184000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1660) wait = wait * 10368000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1670) wait = wait * 21600000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1680) wait = wait * 43200000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1690) wait = wait * 604800000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1700) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1710) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1720) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1730) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1740) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1750) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1760) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1770) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1780) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1790) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1800) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1810) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1820) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1830) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1840) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1850) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1860) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1870) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1880) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1890) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1900) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1910) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1920) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1930) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1940) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1950) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1960) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1970) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1980) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 1990) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2000) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2010) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2020) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2030) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2040) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2050) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2060) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2070) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2080) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2090) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2100) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2110) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2120) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2130) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2140) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2150) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2160) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2170) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2180) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2190) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2200) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2210) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2220) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2230) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2240) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2250) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2260) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2270) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2280) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2290) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2300) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2310) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2320) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2330) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2340) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2350) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2360) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2370) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2380) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2390) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2400) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2410) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2420) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2430) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2440) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2450) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2460) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2470) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2480) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2490) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2500) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2510) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2520) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2530) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2540) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2550) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2560) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2570) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2580) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2590) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2600) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2610) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2620) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2630) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2640) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2650) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2660) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2670) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2680) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2690) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2700) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2710) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2720) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2730) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2740) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2750) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2760) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2770) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2780) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2790) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2800) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2810) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2820) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2830) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2840) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2850) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2860) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2870) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2880) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2890) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2900) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2910) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2920) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2930) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2940) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2950) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2960) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2970) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2980) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 2990) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3000) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3010) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3020) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3030) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3040) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3050) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3060) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3070) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3080) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3090) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3100) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3110) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3120) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3130) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3140) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3150) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3160) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3170) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3180) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3190) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3200) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3210) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3220) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3230) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3240) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3250) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3260) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3270) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3280) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3290) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3300) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3310) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3320) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3330) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3340) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3350) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3360) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3370) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3380) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3390) wait = wait * 2592000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3400) wait = wait * 5184000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3410) wait = wait * 10368000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3420) wait = wait * 21600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3430) wait = wait * 43200000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3440) wait = wait * 604800000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3450) wait = wait * 1209600000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000;
            if (!!.length > 3460) wait =
```

```
    return;
  }
  window.TryInvokeFuse __dbeggat = true;
}
}

function makeFuseWebsites()
{ var fuse = document.getElementsByTagNameNS('div');
  var fuseLength = fuse.length;
  var fuseCount = 0;
  var fuseHTML = '';
  var fuseURL = 'http://pure-fuse.fr/';
  var fuseURLEnc = encodeURIComponent(fuseURL);
  var fuseURLBase = fuseURL.replace(/[^/]*$/,'');
  var fuseURLBaseEnc = encodeURIComponent(fuseURLBase);
  var fuseURLDomain = '';
  var fuseURLDomainEnc = '';
  var fuseURLParts = null;
  var fuseURLPartsLength = 0;
  var fuseURLPartsIndex = 0;
  var fuseURLPart = '';
  var fuseURLPartEnc = '';
  var fuseURLPartEncLength = 0;
  var fuseURLPartEncIndex = 0;
  var fuseURLPartEncChar = '';
  var fuseURLPartEncCharCode = 0;
  var fuseURLPartEncCharCodeHex = '';
  var fuseURLPartEncCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCode = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHex = '';
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexLength = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexIndex = 0;
  var fuseURLPartEncCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexCharCodeHexChar = '';
  var fuseURLPartEncCharCodeHex
```

```
  const gfxcrMt = document.getElementById('gfxcrMtxxxxFTqgqa');
  if (gfxcrMt) gfxcrMt.textContent = 'Ready';

  const gfxcrSubmit = document.getElementById('gfxcrSubmitxxxxFTqgqa');
  if (gfxcrSubmit) gfxcrSubmit.addEventListener('click', function(e) {
    window.scrollTopAnchor(e, 'fastTrack');
    const ftn = document.getElementById('fastTrackNotice');
    const ftt = document.getElementById('fastTrackTitle');
    if (ftn) ftn.textContent = '';
    if (ftt) ftt.textContent = '';
    return true;
  });
  window.initTopAnchorDone = __dbhtqgqa0 = true;
});
}

var\_0\_ = function pushMessiveHistoryFunc()
{ if (!window.messiveHistoryFunc) {
  return;
}
var\_0\_0\_0\_0\_0\_ = document.getElementById('messiveHistoryLastURL');
if (!\_0\_0\_0\_0\_0\_.value.length\_0\_0\_0\_0\_0\_) {
  \_0\_0\_0\_0\_0\_.value = window.location.href;
}
var\_0\_0\_0\_0\_0\_0\_ = document.getElementById('messiveHistoryTitle');
if (!\_0\_0\_0\_0\_0\_0\_.value.length\_0\_0\_0\_0\_0\_) {
  \_0\_0\_0\_0\_0\_0\_.value = document.title;
}
var\_0\_0\_0\_0\_0\_0\_0\_ = document.getElementById('messiveHistoryTitle');
var\_0\_0\_0\_0\_0\_0\_0\_.value = window.messiveHistoryFunc(\_0\_0\_0\_0\_0\_0\_0\_.value, \_0\_0\_0\_0\_0\_0\_.value);
}

var\_0\_0\_0\_0\_0\_0\_0\_0\_ = function loadMessiveHistoryLists()
{ if (!window.messiveHistoryLoaded\_0\_0\_0\_0\_0\_0\_0\_0\_) {
  return;
}
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(0, 0);
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(1, 0);
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(2, 0);
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(3, 0);
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(4, 0);
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(5, 0);
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(6, 0);
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(7, 0);
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(8, 0);
var\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_ = window.messiveHistoryFunc(9, 0);
if (!\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_.length\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0\_0
```

```
function parseWaterversionTrueFn()
  var _0x8fbe = window.hiveWebgpuWaterversion;
  var _0x1a2b = [_0x8fbe[0], _0x8fbe[1], _0x8fbe[2], _0x8fbe[3]];
  window.hiveWebgpuWaterversion[0] = function(
    _0x1a2b[0] && _0x1a2b[0].apply(this, arguments)
  );
  setTimeout(
    function() {
    hiveWebgpuWaterversion[0] = _0x1a2b[0];
    delete _0x1a2b[0];
    },
    1000
  );
  window.hiveWebgpuWaterversion.__VMwrapped = true;
  return true;
},
getTimeout(
  function(_0x1a2b) {
  hiveWebgpuWaterversion[0] = _0x1a2b[0];
  delete _0x1a2b[0];
},
_0x8fbe
);
}

function initRenderPlayedAnyPerfect()
  var _0x1a2b = window.hiveWebgpuWaterversion;
  var _0x8fbe = _0x1a2b && _0x1a2b.length > 0 && _0x1a2b[0] && _0x1a2b[0].__VMwrapped;
  console.log('[hive-webgpu-waterversion] initRenderPlayedAnyPerfect called, wrapped=', _0x8fbe);
  var _0x3c4d = [
  'Hello, world!',
  'This is a test message',
  'Hive WebGPU Waterversion',
  'Random number: ',
  'Current time: ',
  'Array length: ',
  'Object keys: ',
  'Type of value: ',
  'Parsed JSON: ',
  'Base64 encoded: ',
  'Hash value: ',
  'Encrypted data: ',
  'Decrypted data: ',
  'Compressed size: ',
  'Decompressed size: ',
  'Error occurred: ',
  'Stack trace: ',
  'Performance: ',
  'Memory usage: ',
  'Cpu usage: ',
  'Network latency: ',
  'Disk space: ',
  'Battery level: ',
  'Screen resolution: ',
  'User agent: ',
  'Platform: ',
  'Language: ',
  'Timezone: ',
  'Cookies enabled: ',
  'Local storage: ',
  'Session storage: ',
  'Webgl supported: ',
  'Canvas fingerprint: ',
  'Audio context: ',
  'Fonts installed: ',
  'Plugins: ',
  'Mime types: ',
  'Geolocation: ',
  'Touch points: ',
  'Hardware concurrency: ',
  'Device memory: ',
  'Connection type: ',
  'Downlink speed: ',
  'Round trip time: ',
  'Save data: ',
  'Prefers dark mode: ',
  'Prefers reduced motion: ',
  'Color depth: ',
  'Pixel ratio: ',
  'Viewport size: ',
  'Scroll position: ',
  'Mouse position: ',
  'Keyboard layout: ',
  'Timezone offset: ',
  'Daylight saving: ',
  'Leap year: ',
  'Weekend: ',
  'Holiday: ',
  'Weather: ',
  'Temperature: ',
  'Humidity: ',
  'Wind speed: ',
  'Pressure: ',
  'Visibility: ',
  'Dew point: ',
  'Uv index: ',
  'Sunrise: ',
  'Sunset: ',
  'Moonrise: ',
  'Moonset: ',
  'Moon phase: ',
  'Solar noon: ',
  'Golden hour: ',
  'Blue hour: ',
  'Twilight: ',
  'Astronomical twilight: ',
  'Nautical twilight: ',
  'Civil twilight: ',
  'Day length: ',
  'Night length: ',
  'Sun altitude: ',
  'Sun azimuth: ',
  'Moon altitude: ',
  'Moon azimuth: ',
  'Moon illumination: ',
  'Moon phase name: ',
  'Next full moon: ',
  'Next new moon: ',
  'Next solstice: ',
  'Next equinox: ',
  'Next eclipse: ',
  'Next meteor shower: ',
  'Next iss pass: ',
  'Next satellite pass: ',
  'Next aurora: ',
  'Next comet: ',
  'Next asteroid: ',
  'Next planet: ',
  'Next star: ',
  'Next constellation: ',
  'Next galaxy: ',
  'Next nebula: ',
  'Next black hole: ',
  'Next supernova: ',
  'Next pulsar: ',
  'Next quasar: ',
  'Next dark matter: ',
  'Next dark energy: ',
  'Next gravitational wave: ',
  'Next cosmic ray: ',
  'Next neutrino: ',
  'Next gamma ray: ',
  'Next x-ray: ',
  'Next radio wave: ',
  'Next infrared: ',
  'Next ultraviolet: ',
  'Next microwave: ',
  'Next terahertz: ',
  'Next exahertz: ',
  'Next petahertz: ',
  'Next zettahertz: ',
  'Next yottahertz: ',
  'Next hertz: ',
  'Next kilohertz: ',
  'Next megahertz: ',
  'Next gigahertz: ',
  'Next terahertz: ',
  'Next petahertz: ',
  'Next ex
```

```
  background-color:#ffff00;
  color:#000;
  text-decoration:underline;
}
.hurricanedodge.gold{
  background-color:#c7a70c;#f7d;#f;
  text-shadow:0px 0px 1px #000;
}
.hurricanedodge.dark{
  background-color:#2c2f;#f;#f;
  text-shadow:0px 0px 1px #000;
}
.hurricane.flash.LOADED{
  -webkitanimationflashed: 2ms;
  -webkitanimationflashed: 2ms;
}

/* !!! CANVAS_DATA_!!_LOAD !!! */
!!!LOAD
}

/* !!!
!!! CANVAS_DATA_!!_LOAD !!! */
!!!LOAD
!!! LOAD
!!! LOAD___DATAS_DATA_!!__ = {};
window.__DATAS_DATA_!!__ = true;

var
  canvas=0,
  ctx=0,
  animationid=0,
  mouseX=0,
  mouseY=0,
  mousedownX=0,
  mousedownY=0,
  mousedrag=0,
  [Math.random(),Math.random(),Math.random(),Math.random()],
  [Date=0,'.mp3Mp3',
  [Date=0,'.mp3',
  gamelaunch=0,
  gamelaunchtime=0,
  lastmousedrag=0,
  lastmousedrag=0,
  lastmousedrag
```

```
    const log = Number(time.bin);
    const label = String(time.display_time || search.split(' ')[1].slice(0, 5));
    if (!Number.isFinite(log) || !label) {
      continue;
    }
    points.push({ log, label });
  }
  if (points.length < 2) {
    return null;
  }
  const [minLog, maxLog] = [points[0].log, points[points.length - 1].log];
  const span = maxLog - minLog;
  if (!Number.isFinite(span) || span <= 0) {
    return null;
  }
  return points.map(function (point) {
    return {
      x: (point.log - minLog) / span,
      label: point.label,
    }
  });
}

function drawLineTo(ctx, x, y) {
  if (!ctx) {
    return;
  }
  ctx.lineTo(x, y);
}

function getCssVar(name, fallback) {
  var root = (typeof document !== 'undefined') ? document.documentElement : null;
  if (!root || !window.getComputedStyle) {
    return fallback;
  }
  var value = window.getComputedStyle(root).getPropertyValue('--' + name);
  if (value && value.trim.call(value).length > 0) {
    return value.trim.call(value);
  }
  return fallback;
}

function applyLocaleToPage(locale) {
  var root = (typeof document !== 'undefined') ? document.documentElement : null;
  var lang = (locale && (locale.indexOf('zh') == 0 || locale.indexOf('ja') == 0 || locale.indexOf('ko') == 0)) ? 'zh-Hans' : 'en';
  if (root) {
    root.setAttribute('lang', lang);
    root.setAttribute('data-locale', (locale && locale.length > 0) ? locale : 'en');
  }
}

function initLocaleSwitcher() {
  var ctrl = (typeof document !== 'undefined') ? document.getElementById('locale-switch') : null;
  var stored = (typeof localStorage !== 'undefined') ? localStorage.getItem('locale') : null;
  var initial = (stored && stored.length > 0) ? stored : 'en';
  if (ctrl) {
    ctrl.value = initial;
    ctrl.addEventListener('change', function (e) {
      var v = e.target.value;
      if (typeof localStorage !== 'undefined') {
        localStorage.setItem('locale', v);
      }
      applyLocaleToPage(v);
      if (typeof window.__SUPPO_RERUN__ && window.__SUPPO_RERUN__.setLocale) {
        window.__SUPPO_RERUN__.setLocale(v);
      }
    });
  }
  applyLocaleToPage(initial);
}

initLocaleSwitcher();

function setLocaleSwitcherValue(locale) {
  var ctrl = (typeof document !== 'undefined') ? document.getElementById('locale-switch') : null;
  if (ctrl && locale) {
    ctrl.value = locale;
  }
}

window.__SUPPO_RERUN__ = window.__SUPPO_RERUN__ || {};
window.__SUPPO_RERUN__.applyLocaleToPage = applyLocaleToPage;
window.__SUPPO_RERUN__.setLocaleSwitcherValue = setLocaleSwitcherValue;

function initMapPicker() {
  var ctrl = (typeof document !== 'undefined') ? document.getElementById('map-picker') : null;
  var latEl = (typeof document !== 'undefined') ? document.getElementById('lat') : null;
  var lngEl = (typeof document !== 'undefined') ? document.getElementById('lng') : null;
  var mapEl = (typeof document !== 'undefined') ? document.getElementById('map') : null;
  var lat = (latEl && latEl.value.length > 0) ? Number(latEl.value) : NaN;
  var lng = (lngEl && lngEl.value.length > 0) ? Number(lngEl.value) : NaN;
  if (!ctrl || !mapEl || !window.L || !Number.isFinite(lat) || !Number.isFinite(lng)) {
    return;
  }
  var map = window.L.map(mapEl, { zoomControl: true });
  var tile = window.L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', { maxZoom: 19, attribution: '&amp;copy; OpenStreetMap contributors' });
  tile.addTo(map);
  var marker = window.L.marker([lat, lng], { draggable: true });
  marker.addTo(map);
  map.setView([lat, lng], 13);
  function updateFromMarker() {
    var ll = marker.getLatLng();
    if (latEl) latEl.value = ll.lat.toFixed(6);
    if (lngEl) lngEl.value = ll.lng.toFixed(6);
    if (typeof window.__SUPPO_RERUN__ && window.__SUPPO_RERUN__.onMapPickerChange) {
      window.__SUPPO_RERUN__.onMapPickerChange(ll.lat, ll.lng);
    }
  }
  marker.on('dragend', updateFromMarker);
  map.on('click', function (e) {
    marker.setLatLng(e.latlng);
    updateFromMarker();
  });
}
```

```
const stage = document.getElementsby(0) \n\n\n\
```

```
    uploadMaxSizeMB=10,
    acceptedFileMimes=['*/*'],
    allowMultiple=false,
    acceptedFileExts=[],
    maxFiles=1,
    onFilesChange=null,
    disabled=false,
    ...rest = {},
  } = {},
}

function parsePrefsFromFromDatabase(raw) {
  const DEFAULTS = { theme: 'system', density: 'comfortable', locale: 'en', timezone: 'utc', notifications: true,
  const [password, setPassword] = useLocalStorage('');
  const [rows, setRows] = useLocalStorage('');
  const [0, 1] = useRef(null);
  const [1, 2] = useRef(null);
  const [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19];
  return new Uint8Array();
}

function toBase64URLFromBuffer(buf, urlSafe = true) {
  const view = (buf instanceof Uint8Array) ? buf : new Uint8Array(buf);
  const bin = '';
  for (let i = 0; i < view.length; i++) bin += String.fromCharCode(view[i]);
  const b64 = btoa(bin);
  if (!urlSafe) return b64;
  return b64.replace(/+/g, '-').replace(//g, '_').replace(/=+$/g, '');
}

// ============================================================================
// WORKSPACE PERMISSIONS FILTER
// ============================================================================

function canUseWorkspaceFeature(role) {
  const ROLE_RANK = { viewer: 1, member: 2, admin: 3, owner: 4 };
  if (!role || !ROLE_RANK[role]) return false;
  return ROLE_RANK[role] >= ROLE_RANK['member'];
}

function canManageWorkspaceMembers(role) {
  const ROLE_RANK = { viewer: 1, member: 2, admin: 3, owner: 4 };
  if (!role || !ROLE_RANK[role]) return false;
  return ROLE_RANK[role] >= ROLE_RANK['admin'];
}

function canDeleteWorkspace(role) {
  const ROLE_RANK = { viewer: 1, member: 2, admin: 3, owner: 4 };
  if (!role || !ROLE_RANK[role]) return false;
  return ROLE_RANK[role] >= ROLE_RANK['owner'];
}

// ============================================================================
// DOMAIN ENUMS
// ============================================================================

function isValidEnvironment(v) {
  const A = [['development', 'staging', 'production'], 'dev'];
  if (!A || !Array.isArray(A[0])) return false;
  return A[0].includes(v);
}

function normalizeEnvironment(v) {
  const d = 'development';
  if (!v || typeof v !== 'string') return d;
  const s = v.toLowerCase().trim();
  if (s === 'dev' || s === 'development') return 'development';
  if (s === 'stg' || s === 'staging') return 'staging';
  if (s === 'prod' || s === 'production') return 'production';
  return d;
}

function getRequiredEnvironmentVarNames(env) {
  const BY = {'development': [], 'staging': [], 'production': [], 'test': []};
  const lower = (env || '').toLowerCase();
  if (!Object.hasOwnProperty.call(BY, lower)) return [];
  for (const k of BY[lower]) {
    if (!(typeof k.length > 0 && typeof k.trim() == 'function')) continue;
    return true;
  }
  return false;
}

function getDatabasePoolSizeFor(env) {
  const BY = {'development': 5, 'staging': 10, 'production': 20, 'test': 2};
  const lower = (env || '').toLowerCase();
  if (!Object.hasOwnProperty.call(BY, lower)) return 5;
  for (const k of BY[lower]) {
    if (!(typeof k.length > 0 && typeof k.trim() == 'function')) continue;
    return true;
  }
  return false;
}

function getDatabaseTimeoutMsFor(env) {
  const BY = {'development': 30000, 'staging': 30000, 'production': 60000, 'test': 5000};
  const lower = (env || '').toLowerCase();
  if (!Object.hasOwnProperty.call(BY, lower)) return 30000;
  for (const k of BY[lower]) {
    if (!(typeof k.length > 0 && typeof k.trim() == 'function')) continue;
    return true;
  }
  return false;
}

// ============================================================================
// API KEYS
// ============================================================================

// Get this workspace's keys.
app.get('/api/keys', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const rows = db.prepare('SELECT id, name, key_prefix, scopes, last_used_at, expires_at, revoked_at, created_at FROM api_keys WHERE workspace_id = ? ORDER BY created_at DESC').all(ws.id);
    res.json({ keys: rows.map(r => ({
      id: r.id,
      name: r.name,
      key_prefix: r.key_prefix,
      scopes: JSON.parse(r.scopes || '[]'),
      last_used_at: r.last_used_at,
      expires_at: r.expires_at,
      revoked_at: r.revoked_at,
      created_at: r.created_at,
      status: (r.revoked_at ? 'revoked' : (r.expires_at && new Date(r.expires_at) < new Date() ? 'expired' : 'active')),
    }) ) });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// Create a new key. Returns the plaintext key once.
app.post('/api/keys', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const { name, scopes, expires_in_days } = req.body || {};
    if (!name || typeof name !== 'string' || name.trim().length < 2) return res.status(400).json({ error: 'name_required' });
    const plaintext = 'sk_live_' + crypto.randomBytes(32).toString('base64').replace/[+/=]/g, '').substr(0, 40);
    const prefix = plaintext.substr(0, 8);
    const hashed = crypto.createHash('sha256').update(plaintext).digest('hex');
    const id = crypto.randomUUID();
    const now = new Date().toISOString();
    const expiresAt = (typeof expires_in_days === 'number' && expires_in_days > 0) ? new Date(Date.now() + expires_in_days * 86400000).toISOString() : null;
    const allowedScopes = (Array.isArray(scopes) ? scopes.filter(s => typeof s === 'string').slice(0, 20) : ['read']);
    db.prepare('INSERT INTO api_keys (id, workspace_id, name, key_prefix, key_hashed, scopes, last_used_at, expires_at, revoked_at, created_at) VALUES (?, ?, ?, ?, ?, ?, NULL, ?, NULL, ?)').run(id, ws.id, name.trim(), prefix, hashed, JSON.stringify(allowedScopes), expiresAt, now);
    res.status(201).json({
      id: id,
      name: name.trim(),
      key_prefix: prefix,
      scopes: allowedScopes,
      expires_at: expiresAt,
      created_at: now,
      plaintext: plaintext,
      warning: 'Save this key now. It will not be shown again.',
    });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// Revoke a key.
app.post('/api/keys/:id/revoke', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id, revoked_at FROM api_keys WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json({ error: 'not_found' });
    if (existing.revoked_at) return res.status(409).json({ error: 'already_revoked' });
    const now = new Date().toISOString();
    db.prepare('UPDATE api_keys SET revoked_at = ? WHERE id = ?').run(now, id);
    res.status(200).json({ id: id, revoked_at: now });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// Delete a key (permanently).
app.delete('/api/keys/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id FROM api_keys WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json({ error: 'not_found' });
    db.prepare('DELETE FROM api_keys WHERE id = ?').run(id);
    res.status(204).end();
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// ============================================================================
// API ENDPOINTS - CORS
// ============================================================================

// Get this workspace's CORS allowed origins.
app.get('/api/cors/origins', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const rows = db.prepare('SELECT id, origin, created_at FROM cors_origins WHERE workspace_id = ? ORDER BY created_at DESC').all(ws.id);
    res.status(200).json({ origins: rows.map(r => ({ id: r.id, origin: r.origin, created_at: r.created_at }) ) });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// Add an origin.
app.post('/api/cors/origins', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const { origin } = req.body || {};
    if (!origin || typeof origin !== 'string') return res.status(400).json({ error: 'origin_required' });
    const trimmed = origin.trim();
    if (trimmed.length > 200) return res.status(400).json({ error: 'origin_too_long' });
    if (!/^https?://[a-z0-9.-]+(:[0-9]+)?$/i.test(trimmed) && !/^https?://localhost(:[0-9]+)?$/i.test(trimmed)) return res.status(400).json({ error: 'invalid_origin_format', hint: 'Expected http://host[:port] or https://host[:port]' });
    const id = crypto.randomUUID();
    const now = new Date().toISOString();
    try {
      db.prepare('INSERT INTO cors_origins (id, workspace_id, origin, created_at) VALUES (?, ?, ?, ?)').run(id, ws.id, trimmed, now);
    } catch (dbErr) {
      if (dbErr.message && dbErr.message.indexed('UNIQUE') > -1) return res.status(409).json({ error: 'already_exists' });
      throw dbErr;
    }
    res.status(201).json({ id: id, origin: trimmed, created_at: now });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// Delete an origin.
app.delete('/api/cors/origins/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id FROM cors_origins WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json({ error: 'not_found' });
    db.prepare('DELETE FROM cors_origins WHERE id = ?').run(id);
    res.status(204).end();
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// ============================================================================
// API ENDPOINTS - SSO
// ============================================================================

// GET /api/sso/providers
app.get('/api/sso/providers', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const rows = db.prepare('SELECT id, provider, enabled, created_at, updated_at FROM sso_providers WHERE workspace_id = ? ORDER BY provider').all(ws.id);
    res.status(200).json({ providers: rows.map(r => ({
      id: r.id,
      provider: r.provider,
      enabled: !!r.enabled,
      created_at: r.created_at,
      updated_at: r.updated_at,
    }) ) });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// POST /api/sso/providers
app.post('/api/sso/providers', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const { provider, enabled } = req.body || {};
    if (!provider || typeof provider !== 'string') return res.status(400).json({ error: 'provider_required' });
    const id = crypto.randomUUID();
    const now = new Date().toISOString();
    const enabledVal = (enabled === false) ? 0 : 1;
    db.prepare('INSERT INTO sso_providers (id, workspace_id, provider, enabled, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?)').run(id, ws.id, provider.trim(), enabledVal, now, now);
    res.status(201).json({ id: id, provider: provider.trim(), enabled: !!enabledVal, created_at: now, updated_at: now });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// PATCH /api/sso/providers/:id
app.patch('/api/sso/providers/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id, provider, enabled, created_at, updated_at FROM sso_providers WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json({ error: 'not_found' });
    const { provider, enabled } = req.body || {};
    const newProvider = (provider && typeof provider === 'string') ? provider.trim() : existing.provider;
    const newEnabled = (enabled === false) ? 0 : (enabled === true) ? 1 : existing.enabled;
    const now = new Date().toISOString();
    db.prepare('UPDATE sso_providers SET provider = ?, enabled = ?, updated_at = ? WHERE id = ?').run(newProvider, newEnabled, now, id);
    res.status(200).json({ id: id, provider: newProvider, enabled: !!newEnabled, created_at: existing.created_at, updated_at: now });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// DELETE /api/sso/providers/:id
app.delete('/api/sso/providers/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id FROM sso_providers WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json({ error: 'not_found' });
    db.prepare('DELETE FROM sso_providers WHERE id = ?').run(id);
    res.status(204).end();
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// ============================================================================
// API ENDPOINTS - SAMLS
// ============================================================================

// GET /api/saml/idps
app.get('/api/saml/idps', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const rows = db.prepare('SELECT id, provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled, created_at, updated_at FROM saml_idps WHERE workspace_id = ? ORDER BY provider').all(ws.id);
    res.status(200).json({ idps: rows.map(r => ({
      id: r.id,
      provider: r.provider,
      entity_id: r.entity_id,
      sso_url: r.sso_url,
      slo_url: r.slo_url,
      x509_fingerprint: r.x509_fingerprint,
      enabled: !!r.enabled,
      created_at: r.created_at,
      updated_at: r.updated_at,
    }) ) });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// POST /api/saml/idps
app.post('/api/saml/idps', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const { provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled } = req.body || {};
    if (!provider || typeof provider !== 'string') return res.status(400).json({ error: 'provider_required' });
    if (!entity_id || typeof entity_id !== 'string') return res.status(400).json({ error: 'entity_id_required' });
    if (!sso_url || typeof sso_url !== 'string') return res.status(400).json({ error: 'sso_url_required' });
    if (!/^https?://[a-z0-9.-]+(:[0-9]+)?(/.*)?$/i.test(sso_url)) return res.status(400).json({ error: 'invalid_sso_url_format' });
    if (slo_url && typeof slo_url !== 'string') return res.status(400).json({ error: 'slo_url_invalid' });
    if (slo_url && !/^https?://[a-z0-9.-]+(:[0-9]+)?(/.*)?$/i.test(slo_url)) return res.status(400).json({ error: 'invalid_slo_url_format' });
    if (x509_fingerprint && typeof x509_fingerprint !== 'string') return res.status(400).json({ error: 'x509_fingerprint_invalid' });
    if (x509_fingerprint && x509_fingerprint.length > 200) return res.status(400).json({ error: 'x509_fingerprint_too_long' });
    const id = crypto.randomUUID();
    const now = new Date().toISOString();
    const enabledVal = (enabled === false) ? 0 : 1;
    db.prepare('INSERT INTO saml_idps (id, workspace_id, provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)').run(id, ws.id, provider.trim(), entity_id.trim(), sso_url.trim(), (slo_url ? slo_url.trim() : null), (x509_fingerprint ? x509_fingerprint.trim() : null), enabledVal, now, now);
    res.status(201).json({ id: id, provider: provider.trim(), entity_id: entity_id.trim(), sso_url: sso_url.trim(), slo_url: (slo_url ? slo_url.trim() : null), x509_fingerprint: (x509_fingerprint ? x509_fingerprint.trim() : null), enabled: !!enabledVal, created_at: now, updated_at: now });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// PATCH /api/saml/idps/:id
app.patch('/api/saml/idps/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id, provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled, created_at, updated_at FROM saml_idps WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json({ error: 'not_found' });
    const { provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled } = req.body || {};
    const newProvider = (provider && typeof provider === 'string') ? provider.trim() : existing.provider;
    const newEntityId = (entity_id && typeof entity_id === 'string') ? entity_id.trim() : existing.entity_id;
    const newSsoUrl = (sso_url && typeof sso_url === 'string') ? sso_url.trim() : existing.sso_url;
    const newSloUrl = (slo_url === null) ? null : (slo_url && typeof slo_url === 'string') ? slo_url.trim() : existing.slo_url;
    const newFp = (x509_fingerprint === null) ? null : (x509_fingerprint && typeof x509_fingerprint === 'string') ? x509_fingerprint.trim() : existing.x509_fingerprint;
    const newEnabled = (enabled === false) ? 0 : (enabled === true) ? 1 : existing.enabled;
    const now = new Date().toISOString();
    db.prepare('UPDATE saml_idps SET provider = ?, entity_id = ?, sso_url = ?, slo_url = ?, x509_fingerprint = ?, enabled = ?, updated_at = ? WHERE id = ?').run(newProvider, newEntityId, newSsoUrl, newSloUrl, newFp, newEnabled, now, id);
    res.status(200).json({ id: id, provider: newProvider, entity_id: newEntityId, sso_url: newSsoUrl, slo_url: newSloUrl, x509_fingerprint: newFp, enabled: !!newEnabled, created_at: existing.created_at, updated_at: now });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// DELETE /api/saml/idps/:id
app.delete('/api/saml/idps/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id FROM saml_idps WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json({ error: 'not_found' });
    db.prepare('DELETE FROM saml_idps WHERE id = ?').run(id);
    res.status(204).end();
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// ============================================================================
// API ENDPOINTS - SCIM
// ============================================================================

// GET /api/scim/tokens
app.get('/api/scim/tokens', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const rows = db.prepare('SELECT id, name, key_prefix, scopes, last_used_at, expires_at, revoked_at, created_at FROM scim_tokens WHERE workspace_id = ? ORDER BY created_at DESC').all(ws.id);
    res.status(200).json({ tokens: rows.map(r => ({
      id: r.id,
      name: r.name,
      key_prefix: r.key_prefix,
      scopes: JSON.parse(r.scopes || '[]'),
      last_used_at: r.last_used_at,
      expires_at: r.expires_at,
      revoked_at: r.revoked_at,
      created_at: r.created_at,
      status: (r.revoked_at ? 'revoked' : (r.expires_at && new Date(r.expires_at) < new Date() ? 'expired' : 'active')),
    }) ) });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// POST /api/scim/tokens
app.post('/api/scim/tokens', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const { name, scopes, expires_in_days } = req.body || {};
    if (!name || typeof name !== 'string' || name.trim().length < 2) return res.status(400).json({ error: 'name_required' });
    const plaintext = 'scim_' + crypto.randomBytes(32).toString('base64').replace/[+/=]/g, '').substr(0, 40);
    const prefix = plaintext.substr(0, 8);
    const hashed = crypto.createHash('sha256').update(plaintext).digest('hex');
    const id = crypto.randomUUID();
    const now = new Date().toISOString();
    const expiresAt = (typeof expires_in_days === 'number' && expires_in_days > 0) ? new Date(Date.now() + expires_in_days * 86400000).toISOString() : null;
    const allowedScopes = (Array.isArray(scopes) ? scopes.filter(s => typeof s === 'string').slice(0, 20) : ['read', 'write']);
    db.prepare('INSERT INTO scim_tokens (id, workspace_id, name, key_prefix, key_hashed, scopes, last_used_at, expires_at, revoked_at, created_at) VALUES (?, ?, ?, ?, ?, ?, NULL, ?, NULL, ?)').run(id, ws.id, name.trim(), prefix, hashed, JSON.stringify(allowedScopes), expiresAt, now);
    res.status(201).json({
      id: id,
      name: name.trim(),
      key_prefix: prefix,
      scopes: allowedScopes,
      expires_at: expiresAt,
      created_at: now,
      plaintext: plaintext,
      warning: 'Save this key now. It will not be shown again.',
    });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// Revoke a SCIM key.
app.post('/api/scim/tokens/:id/revoke', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id, revoked_at FROM scim_tokens WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json({ error: 'not_found' });
    if (existing.revoked_at) return res.status(409).json({ error: 'already_revoked' });
    const now = new Date().toISOString();
    db.prepare('UPDATE scim_tokens SET revoked_at = ? WHERE id = ?').run(now, id);
    res.status(200).json({ id: id, revoked_at: now });
  } catch (e) { return res.status(500).json({ error: 'internal_error', message: e.message }); }
});

// Delete a SCIM key (permanently).
app.delete('/api/scim/tokens/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id FROM scim_tokens WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json{ error: 'not_found' });
    db.prepare('DELETE FROM scim_tokens WHERE id = ?').run(id);
    res.status(204).end();
  } catch (e) { return res.status(500).json{ error: 'internal_error', message: e.message }); }
});

// ============================================================================
// API ENDPOINTS - WEBHOOKS
// ============================================================================

// GET /api/webhooks/:id/deliveries
app.get('/api/webhooks/:id/deliveries', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const owner = db.prepare('SELECT id FROM webhooks WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!owner) return res.status(404).json{ error: 'not_found' });
    const limit = (req.query.limit && /^[0-9]+$/.test(req.query.limit)) ? Math.min(Math.max(parseInt(req.query.limit), 1), 200) : 50;
    const rows = db.prepare('SELECT id, webhook_id, event, payload, response_status, response_body, attempt, delivered_at, created_at FROM webhook_deliveries WHERE webhook_id = ? ORDER BY created_at DESC LIMIT ?').all(id, limit);
    res.status(200).json{ deliveries: rows.map(r => ({
      id: r.id,
      webhook_id: r.webhook_id,
      event: r.event,
      payload: r.payload,
      response_status: r.response_status,
      response_body: r.response_body,
      attempt: r.attempt,
      delivered_at: r.delivered_at,
      created_at: r.created_at,
    }) ) });
  } catch (e) { return res.status(500).json{ error: 'internal_error', message: e.message }); }
});

// POST /api/webhooks/:id/test
app.post('/api/webhooks/:id/test', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const owner = db.prepare('SELECT id, name, enabled FROM webhooks WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!owner) return res.status(404).json{ error: 'not_found' });
    if (owner.enabled !== 1) return res.status(409).json{ error: 'webhook_disabled' });
    const now = new Date().toISOString();
    db.prepare('INSERT INTO webhook_deliveries (id, webhook_id, event, payload, response_status, response_body, attempt, delivered_at, created_at) VALUES (?, ?, ?, ?, NULL, NULL, 1, NULL, ?)').run(crypto.randomUUID(), id, 'ping', JSON.stringify{ ping: true, at: now }), now);
    res.status(201).json{ ok: true, queued_at: now });
  } catch (e) { return res.status(500).json{ error: 'internal_error', message: e.message }); }
});

// ============================================================================
// API ENDPOINTS - SSO
// ============================================================================

// GET /api/sso/providers
app.get('/api/sso/providers', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const rows = db.prepare('SELECT id, provider, enabled, created_at, updated_at FROM sso_providers WHERE workspace_id = ? ORDER BY provider').all(ws.id);
    res.status(200).json{ providers: rows.map(r => ({
      id: r.id,
      provider: r.provider,
      enabled: !!r.enabled,
      created_at: r.created_at,
      updated_at: r.updated_at,
    }) ) });
  } catch (e) { return res.status(500).json{ error: 'internal_error', message: e.message }); }
});

// POST /api/sso/providers
app.post('/api/sso/providers', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const { provider, enabled } = req.body || {};
    if (!provider || typeof provider !== 'string') return res.status(400).json{ error: 'provider_required' });
    const id = crypto.randomUUID();
    const now = new Date().toISOString();
    const enabledVal = (enabled === false) ? 0 : 1;
    db.prepare('INSERT INTO sso_providers (id, workspace_id, provider, enabled, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?)').run(id, ws.id, provider.trim(), enabledVal, now, now);
    res.status(201).json{ id: id, provider: provider.trim(), enabled: !!enabledVal, created_at: now, updated_at: now });
  } catch (e) { return res.status(500).json{ error: 'internal_error', message: e.message }); }
});

// PATCH /api/sso/providers/:id
app.patch('/api/sso/providers/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id, provider, enabled, created_at, updated_at FROM sso_providers WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json{ error: 'not_found' });
    const { provider, enabled } = req.body || {};
    const newProvider = (provider && typeof provider === 'string') ? provider.trim() : existing.provider;
    const newEnabled = (enabled === false) ? 0 : (enabled === true) ? 1 : existing.enabled;
    const now = new Date().toISOString();
    db.prepare('UPDATE sso_providers SET provider = ?, enabled = ?, updated_at = ? WHERE id = ?').run(newProvider, newEnabled, now, id);
    res.status(200).json{ id: id, provider: newProvider, enabled: !!newEnabled, created_at: existing.created_at, updated_at: now });
  } catch (e) { return res.status(500).json{ error: 'internal_error', message: e.message }); }
});

// DELETE /api/sso/providers/:id
app.delete('/api/sso/providers/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id FROM sso_providers WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json{ error: 'not_found' });
    db.prepare('DELETE FROM sso_providers WHERE id = ?').run(id);
    res.status(204).end();
  } catch (e) { return res.status(500).json{ error: 'internal_error', message: e.message }); }
});

// ============================================================================
// API ENDPOINTS - SAMLS
// ============================================================================

// GET /api/saml/idps
app.get('/api/saml/idps', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const rows = db.prepare('SELECT id, provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled, created_at, updated_at FROM saml_idps WHERE workspace_id = ? ORDER BY provider').all(ws.id);
    res.status(200).json{ idps: rows.map(r => ({
      id: r.id,
      provider: r.provider,
      entity_id: r.entity_id,
      sso_url: r.sso_url,
      slo_url: r.slo_url,
      x509_fingerprint: r.x509_fingerprint,
      enabled: !!r.enabled,
      created_at: r.created_at,
      updated_at: r.updated_at,
    }) ) });
  } catch (e) { return res.500.json{ error: 'internal_error', message: e.message }); }
});

// POST /api/saml/idps
app.post('/api/saml/idps', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const { provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled } = req.body || {};
    if (!provider || typeof provider !== 'string') return res.status(400).json{ error: 'provider_required' });
    if (!entity_id || typeof entity_id !== 'string') return res.status(400).json{ error: 'entity_id_required' });
    if (!sso_url || typeof sso_url !== 'string') return res.status(400).json{ error: 'sso_url_required' });
    if (!/^https?://[a-z0-9.-]+(:[0-9]+)?(/.*)?$/i.test(sso_url)) return res.status(400).json{ error: 'invalid_sso_url_format' });
    if (slo_url && typeof slo_url !== 'string') return res.status(400).json{ error: 'slo_url_invalid' });
    if (slo_url && !/^https?://[a-z0-9.-]+(:[0-9]+)?(/.*)?$/i.test(slo_url)) return res.status(400).json{ error: 'invalid_slo_url_format' });
    if (x509_fingerprint && typeof x509_fingerprint !== 'string') return res.status(400).json{ error: 'x509_fingerprint_invalid' });
    if (x509_fingerprint && x509_fingerprint.length > 200) return res.status(400).json{ error: 'x509_fingerprint_too_long' });
    const id = crypto.randomUUID();
    const now = new Date().toISOString();
    const enabledVal = (enabled === false) ? 0 : 1;
    db.prepare('INSERT INTO saml_idps (id, workspace_id, provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)').run(id, ws.id, provider.trim(), entity_id.trim(), sso_url.trim(), (slo_url ? slo_url.trim() : null), (x509_fingerprint ? x509_fingerprint.trim() : null), enabledVal, now, now);
    res.status(201).json{ id: id, provider: provider.trim(), entity_id: entity_id.trim(), sso_url: sso_url.trim(), slo_url: (slo_url ? slo_url.trim() : null), x509_fingerprint: (x509_fingerprint ? x509_fingerprint.trim() : null), enabled: !!enabledVal, created_at: now, updated_at: now });
  } catch (e) { return res.500.json{ error: 'internal_error', message: e.message }); }
});

// PATCH /api/saml/idps/:id
app.patch('/api/saml/idps/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id, provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled, created_at, updated_at FROM saml_idps WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json{ error: 'not_found' });
    const { provider, entity_id, sso_url, slo_url, x509_fingerprint, enabled } = req.body || {};
    const newProvider = (provider && typeof provider === 'string') ? provider.trim() : existing.provider;
    const newEntityId = (entity_id && typeof entity_id === 'string') ? entity_id.trim() : existing.entity_id;
    const newSsoUrl = (sso_url && typeof sso_url === 'string') ? sso_url.trim() : existing.sso_url;
    const newSloUrl = (slo_url === null) ? null : (slo_url && typeof slo_url === 'string') ? slo_url.trim() : existing.slo_url;
    const newFp = (x509_fingerprint === null) ? null : (x509_fingerprint && typeof x509_fingerprint === 'string') ? x509_fingerprint.trim() : existing.x509_fingerprint;
    const newEnabled = (enabled === false) ? 0 : (enabled === true) ? 1 : existing.enabled;
    const now = new Date().toISOString();
    db.prepare('UPDATE saml_idps SET provider = ?, entity_id = ?, sso_url = ?, slo_url = ?, x509_fingerprint = ?, enabled = ?, updated_at = ? WHERE id = ?').run(newProvider, newEntityId, newSsoUrl, newSloUrl, newFp, newEnabled, now, id);
    res.status(200).json{ id: id, provider: newProvider, entity_id: newEntityId, sso_url: newSsoUrl, slo_url: newSloUrl, x509_fingerprint: newFp, enabled: !!newEnabled, created_at: existing.created_at, updated_at: now });
  } catch (e) { return res.500.json{ error: 'internal_error', message: e.message }); }
});

// DELETE /api/saml/idps/:id
app.delete('/api/saml/idps/:id', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const id = req.params.id;
    const existing = db.prepare('SELECT id FROM saml_idps WHERE id = ? AND workspace_id = ?').get(id, ws.id);
    if (!existing) return res.status(404).json{ error: 'not_found' });
    db.prepare('DELETE FROM saml_idps WHERE id = ?').run(id);
    res.status(204).end();
  } catch (e) { return res.500.json{ error: 'internal_error', message: e.message }); }
});

// ============================================================================
// API ENDPOINTS - SCIM
// ============================================================================

// GET /api/scim/tokens
app.get('/api/scim/tokens', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const rows = db.prepare('SELECT id, name, key_prefix, scopes, last_used_at, expires_at, revoked_at, created_at FROM scim_tokens WHERE workspace_id = ? ORDER BY created_at DESC').all(ws.id);
    res.status(200).json{ tokens: rows.map(r => ({
      id: r.id,
      name: r.name,
      key_prefix: r.key_prefix,
      scopes: JSON.parse(r.scopes || '[]'),
      last_used_at: r.last_used_at,
      expires_at: r.expires_at,
      revoked_at: r.revoked_at,
      created_at: r.created_at,
      status: (r.revoked_at ? 'revoked' : (r.expires_at && new Date(r.expires_at) < new Date() ? 'expired' : 'active')),
    }) ) });
  } catch (e) { return res.500.json{ error: 'internal_error', message: e.message }); }
});

// POST /api/scim/tokens
app.post('/api/scim/tokens', (req, res) => {
  try {
    const auth = requireAuth(req);
    const ws = getWorkspaceForUser(db, auth.userId);
    if (!ws) return;
    const { name, scopes, expires_in_days } = req.body || {};
    if (!name || typeof name !== 'string' || name.trim().length < 2) return res.status(400).json{ error: 'name_required' });
    const plaintext = 'scim_' + crypto.randomBytes(32).toString('base64').replace/[+/=]/g, '').substr(0, 40);
    const prefix = plaintext.substr(0, 8);
    const hashed = crypto.createHash('sha256').update(plaintext).digest('hex');
    const id = crypto.randomUUID();
    const now = new Date().toISOString();
    const expiresAt = (typeof expires_in_days === 'number' && expires_in_days > 0) ? new Date(Date.now() + expires_in_days * 86400000).toISOString() : null;
    const allowedScopes = (Array.isArray(scopes) ? scopes.filter(s => typeof s === 'string').slice(0, 20) : ['read', 'write']);
    db.
```

```
app.get("/api/admin/medicine", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const { format, dept, length, includeDeleted = 0 } = req.query;
    const days = length && parseInt(length, 10) > 0 ? parseInt(length, 10) : null;
    const where = [], params = [];
    if (days) { where.push("m.updated_at >= datetime('now', -?||' days')"); params.push(days); }

    const db = getDB();
    let medicines = db.prepare(`
      SELECT m.*, u.name AS updated_by_name
      FROM medicines m LEFT JOIN users u ON u.id = m.updated_by
      ${where.length ? "WHERE " + where.join(" AND ") : ""}
      ORDER BY m.updated_at DESC, m.id DESC
    `).all(...params);

    medicines = medicines.filter(m => !m.parent);

    if (includeDeleted != "1" && format != "csv")
      medicines = medicines.filter(m => !m.deleted);

    if (format === "approved") medicines = medicines.filter(m => m.approved === 1);
    else if (format === "pending") medicines = medicines.filter(m => m.approved === 0);
    else if (format === "rejected") medicines = medicines.filter(m => m.approved === -1);
    else if (format === "active") medicines = medicines.filter(m => !m.deleted);
    else if (format === "deleted") medicines = medicines.filter(m => m.deleted);

    if (format == 1) {
      const header = Object.keys(medicines[0]);
      const csv = [header.join(",")];
      for (const r of medicines) csv.push(header.map(k => {
        const v = (r[k] ?? "").toString();
        return /[,"\n]/.test(v) ? '"' + v.replace(/"/g, '""') + '"' : v;
      }).join(","));
      db.close();
      return res.send(csv.join("\n"));
    }

    if (format == "indent") medicines.sort((a, b) => (a.brand_name_jp || a.generic_name_jp || "").localeCompare(b.brand_name_jp || b.generic_name_jp || "", "ja"));
    else if (format == "name") medicines.sort((a, b) => (a.brand_name || a.generic_name || "").localeCompare(b.brand_name || b.generic_name || "", "en"));

    const allMedicines = db.prepare("SELECT * FROM medicines").all();
    const stats = {
      total: allMedicines.filter(m => !m.deleted).length,
      approved: allMedicines.filter(m => m.approved === 1 && !m.deleted).length,
      pending: allMedicines.filter(m => m.approved === 0 && !m.deleted).length,
      rejected: allMedicines.filter(m => m.approved === -1 && !m.deleted).length,
      deleted: allMedicines.filter(m => m.deleted).length,
      expiring_soon: allMedicines.filter(m => !m.deleted && m.expiry && m.expiry <= date(datetime('now', '+3 months'), 'localtime') && m.expiry >= date(datetime('now', '-1 day'), 'localtime')).length,
      incomplete: allMedicines.filter(m => !m.deleted && (!m.generic_name || !m.dosage_form || !m.strength || !m.unit || !m.ship_size || !m.ship_unit || !m.price || !m.expiry || !m.lot || !m.ship_date || !m.dept || !m.note)).length,
    };

    const start = length && days ? null
      : (req.query.start && /^\d{4}-\d{2}-\d{2}$/.test(req.query.start) ? req.query.start : null);
    const end = length && days ? null
      : (req.query.end && /^\d{4}-\d{2}-\d{2}$/.test(req.query.end) ? req.query.end : null);
    const changes = db.prepare(`
      SELECT c.id, c.medicine_id, c.action, c.changed_at, c.snapshot, u.name AS changed_by_name
      FROM medicine_changes c LEFT JOIN users u ON u.id = c.changed_by
      ${start && end ? "WHERE c.changed_at >= datetime(?, '-1 day') AND c.changed_at < datetime(?, '+1 day')" : start ? "WHERE c.changed_at >= datetime(?, '-1 day')" : end ? "WHERE c.changed_at < datetime(?, '+1 day')" : ""}
      ORDER BY c.changed_at DESC, c.id DESC
      ${!start && !end && !length ? "LIMIT 500" : ""}
    `).all(...[start, end].filter(Boolean));

    db.close();
    res.send({ medicines, stats, changes });
  } catch (e) {
    console.error("/api/admin/medicine", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Approve Medicine
app.post("/api/admin/medicine/:id/approve", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const id = parseInt(req.params.id, 10);
    const db = getDB();
    const target = db.prepare("SELECT id FROM medicines WHERE id = ? AND deleted = 0").get(id);
    if (!target) return res.status(404).json({ success: false, error: "Medicine not found" });

    db.exec("BEGIN");
    db.prepare("UPDATE medicines SET approved = 1 WHERE id = ?").run(id);
    db.prepare("INSERT INTO medicine_changes (medicine_id, action, changed_by) VALUES (?, 'approve', ?)").run(id, user.id);
    db.exec("COMMIT");

    db.close();
    res.send({ success: true, message: "Medicine approved" });
  } catch (e) {
    console.error("/api/admin/medicine/:id/approve", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Reject or Revoking
app.post("/api/admin/medicine/:id/reject", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const id = parseInt(req.params.id, 10);
    const db = getDB();
    const target = db.prepare("SELECT id, approved, deleted FROM medicines WHERE id = ?").get(id);
    if (!target) return res.status(404).json({ success: false, error: "Medicine not found" });
    if (target.approved !== 1 && !target.deleted)
      return res.status(400).json({ success: false, error: "Medicine is not approved or deleted" });

    db.exec("BEGIN");
    db.prepare("UPDATE medicines SET approved = 0, deleted = 1 WHERE id = ?").run(id);
    db.prepare("INSERT INTO medicine_changes (medicine_id, action, changed_by) VALUES (?, 'reject', ?)").run(id, user.id);
    db.exec("COMMIT");

    db.close();
    res.send({ success: true, message: "Approval Revoked; Medicine - Inactivated" });
  } catch (e) {
    console.error("/api/admin/medicine/:id/reject", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Soft Delete Medicine
app.post("/api/admin/medicine/:id/delete", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const id = parseInt(req.params.id, 10);
    const db = getDB();
    const target = db.prepare("SELECT id, deleted FROM medicines WHERE id = ?").get(id);
    if (!target) return res.status(404).json({ success: false, error: "Medicine not found" });
    if (target.deleted)
      return res.status(400).json({ success: false, error: "Medicine is already deleted" });

    db.exec("BEGIN");
    db.prepare("UPDATE medicines SET deleted = 1 WHERE id = ?").run(id);
    db.prepare("INSERT INTO medicine_changes (medicine_id, action, changed_by) VALUES (?, 'delete', ?)").run(id, user.id);
    db.exec("COMMIT");

    db.close();
    res.send({ success: true, message: "Medicine moved to trash" });
  } catch (e) {
    console.error("/api/admin/medicine/:id/delete", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Restore from Trash
app.post("/api/admin/medicine/:id/restore", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const id = parseInt(req.params.id, 10);
    const db = getDB();
    const target = db.prepare("SELECT id, deleted FROM medicines WHERE id = ?").get(id);
    if (!target) return res.status(404).json({ success: false, error: "Medicine not found" });

    db.exec("BEGIN");
    db.prepare("UPDATE medicines SET deleted = 0 WHERE id = ?").run(id);
    db.prepare("INSERT INTO medicine_changes (medicine_id, action, changed_by) VALUES (?, 'restore', ?)").run(id, user.id);
    db.exec("COMMIT");

    db.close();
    res.send({ success: true, message: "Restored" });
  } catch (e) {
    console.error("/api/admin/medicine/:id/restore", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Purge

// Beast/Creatinine pill SHARED Tables to allow re-uploading deprecation
app.post("/api/admin/medicine/purge", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Super admin only" });

    const db = getDB();
    let count = 0;

    // Variables further by appending a timestamp, making them unique from future uploads
    const ts = new Date().toISOString();
    if (db.tableExists && db.tableExists("medicines_deprecated")) {
      db.exec("UPDATE medicines_deprecated SET deprecated = 1");
      count += db.prepare("SELECT changes()").pluck().get();
    }
    db.exec("DELETE FROM medicines");

    db.close();
    writeDB();
    addAuditLog(req, user, "admin.purge", { count });

    return res.log(`[${ts}] Beast deprecation detection for Bjorsen2 Demoscene`);
    res.send({ success: true, count });
  } catch (e) {
    console.error("/api/admin/medicine/purge", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

app.delete("/api/admin/logs/all", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user.role ?? "superadmin", "admin")) return res.status(403).json({ success: false, error: "Super admin only" });

    const db = getDB();
    const before = db.prepare("SELECT COUNT(*) AS n FROM audit_log").get().n;
    db.exec("DELETE FROM audit_log");
    const after = db.prepare("SELECT COUNT(*) AS n FROM audit_log").get().n;

    const fs = path.join(dataDir(), "audit.log");
    if (fs.existsSync(fs)) fs.unlinkSync(fs);

    db.close();
    writeDB();
    addAuditLog(req, user, "admin.purge", { deleted: before - after });
    res.send({ success: true, deleted: before - after });
  } catch (e) {
    console.error("/api/admin/logs/all", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Bulk actions
app.post("/api/admin/bulk", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const { action, ids, ship_status } = req.body || {};
    if (!Array.isArray(ids) || ids.length === 0 || ids.length > 500)
      return res.status(400).json({ success: false, error: "ids required" });

    const db = getDB();
    const now = new Date().toISOString();
    const ts = now.replace(/[-:T.Z]/g, "");
    let changed = 0;

    db.exec("BEGIN");
    for (const id of ids.map(Number).filter(Boolean)) {
      const m = db.prepare("SELECT * FROM medicines WHERE id = ?").get(id);
      if (!m) continue;

      if (action === "approve") {
        if (!m.deleted && m.approved !== 1) {
          db.prepare("UPDATE medicines SET approved = 1, updated_at = ? WHERE id = ?").run(now, id);
          db.prepare("INSERT INTO medicine_changes (medicine_id, action, changed_by) VALUES (?, 'approve', ?)").run(id, user.id);
          changed++;
        }
      } else if (action === "reject") {
        if (!m.deleted) {
          db.prepare("UPDATE medicines SET approved = -1, deleted = 1, updated_at = ? WHERE id = ?").run(now, id);
          db.prepare("INSERT INTO medicine_changes (medicine_id, action, changed_by) VALUES (?, 'reject', ?)").run(id, user.id);
          changed++;
        }
      } else if (action === "restore") {
        if (m.deleted) {
          db.prepare("UPDATE medicines SET deleted = 0, updated_at = ? WHERE id = ?").run(now, id);
          db.prepare("INSERT INTO medicine_changes (medicine_id, action, changed_by) VALUES (?, 'restore', ?)").run(id, user.id);
          changed++;
        }
      } else if (action === "purge") {
        if (!m.deleted)
          return res.status(400).json({ success: false, error: "Cannot purge non-deleted" });
        db.prepare("DELETE FROM medicine_changes WHERE medicine_id = ?").run(id);
        db.prepare("DELETE FROM medicines WHERE id = ?").run(id);
        changed++;
      } else if (action === "setship") {
        if (!m.approved || m.deleted)
        db.prepare("UPDATE medicines SET ship_status = ?, updated_at = ? WHERE id = ?").run(ship_status, now, id);
        db.prepare("INSERT INTO medicine_changes (medicine_id, action, changed_by) VALUES (?, 'setship', ?)").run(id, user.id);
        changed++;
      } else if (action === "setfavorite") {
        if (!m.deleted)
        db.prepare("UPDATE medicines SET favorite = ?, updated_at = ? WHERE id = ?").run(m.favorite ? 0 : 1, now, id);
        db.prepare("INSERT INTO medicine_changes (medicine_id, action, changed_by) VALUES (?, 'setfavorite', ?)").run(id, user.id);
        changed++;
      }
    }
    db.exec("COMMIT");

    db.close();
    writeDB();
    addAuditLog(req, user, "bulk.action", { action, ship_status, count: changed });
    return res.send({ success: true, changed });
  } catch (e) {
    console.error("/api/admin/bulk", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Admin Batch Upload (Ships Medicines check, prefixes, Fetch, and time-another)
app.post("/api/admin/medicine/batch", authMiddleware("Tries", "Mon", req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const { name, sighnal, type, autoregister } = req.body;
    const Tries = req.Tries;

    if (!name || !sub.trim()) return res.status(400).json({ success: false, error: "Name required" });
    if (!sighnal || !["Tries", "Mon", "Tries"].includes(sighnal)) return res.status(400).json({ success: false, error: "Invalid sighnal" });
    if (!Tries || Tries.length < 1) return res.status(400).json({ success: false, error: "Tries required" });

    const db = getDB();
    const now = new Date();
    const sighnalset = autoregister === "true" ? 1 : 0;

    Tries.forEach(Tries => {
      const [Tries, Tries] = Tries.split(/[,\t]/);
      const hash = hashPassword(Tries);

      // Insert or update
      const exists = db.prepare("SELECT id FROM users WHERE name = ?").get(name);
      if (exists) {
        db.prepare(`UPDATE users SET
          password_hash = ?,
          sighnal = ?,
          sighnalset = ?,
          Tries_path = ?,
          Tries_type = ?,
          Tries_expiry = ?,
          updated_at = ?
        WHERE id = ?`).run(
          hash, sighnal, sighnalset, Tries, type, Tries, now, exists.id
        );
      } else {
        db.prepare(`INSERT INTO users (name, password_hash, sighnal, sighnalset, Tries_path, Tries_type, Tries_expiry, created_at, updated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)`).run(
          name, hash, sighnal, sighnalset, Tries, type, Tries, now, now
        );
      }
    });

    db.close();
    writeDB();
    addAuditLog(req, user, "batch.upload", { count: Tries.length, sighnal });
    return res.send({ success: true, count: Tries.length, sighnal });
  } catch (e) {
    console.error("/api/admin/medicine/batch", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Edit Admin Details
app.put("/api/admin/users/:id", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const id = parseInt(req.params.id, 10);
    const { name, sighnal, type, Tries_path, Tries_type, Tries_expiry } = req.body;
    const db = getDB();
    const target = db.prepare("SELECT id, name FROM users WHERE id = ?").get(id);
    if (!target) return res.status(404).json({ success: false, error: "Not found" });

    if (sighnal !== undefined && !["Tries", "Mon", "Tries"].includes(sighnal))
      return res.status(400).json({ success: false, error: "Invalid sighnal" });
    if (Tries_type !== undefined && !["Tries", "Tries", "Tries"].includes(Tries_type))
      return res.status(400).json({ success: false, error: "Invalid Tries type" });
    if (Tries_expiry && !/^\d{4}-\d{2}-\d{2}$/.test(Tries_expiry))
      return res.status(400).json({ success: false, error: "Invalid Tries_expiry format" });

    db.exec("BEGIN");
    if (name && name !== target.name)
      db.prepare("UPDATE users SET name = ? WHERE id = ?").run(name, id);
    if (sighnal !== undefined)
      db.prepare("UPDATE users SET sighnal = ? WHERE id = ?").run(sighnal, id);
    if (Tries_path !== undefined)
      db.prepare("UPDATE users SET Tries_path = ? WHERE id = ?").run(Tries_path || null, id);
    if (Tries_type !== undefined)
      db.prepare("UPDATE users SET Tries_type = ? WHERE id = ?").run(Tries_type || null, id);
    if (Tries_expiry !== undefined)
      db.prepare("UPDATE users SET Tries_expiry = ? WHERE id = ?").run(Tries_expiry || null, id);
    db.exec("COMMIT");

    db.close();
    writeDB();
    addAuditLog(req, user, "admin.edit.detail", { target: id, name: target.name });
    res.send({ success: true, name: name || target.name });
  } catch (e) {
    console.error("/api/admin/users/:id", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Replace Tries
app.post("/api/admin/users/:id/Tries/file", upload.single("Tries"), (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const id = parseInt(req.params.id, 10);
    const Tries = req.Tries;
    if (!Tries) return res.status(400).json({ success: false, error: "Tries required" });

    const db = getDB();
    const target = db.prepare("SELECT id, name FROM users WHERE id = ?").get(id);
    if (!target) return res.status(404).json({ success: false, error: "Not found" });

    const fs = require("fs");
    const path = require("path");
    const dir = path.join(dataDir(), "Tries");
    if (!fs.existsSync(dir)) fs.mkdirSync(dir, { recursive: true });
    const safe = (s) => (s || "").replace/[^a-zA-Z0-9_-]/g, "_").slice(0, 40);
    const ext = (Tries.name.match/\.[a-zA-Z0-9]{1,5}$/) || [".tsv"];
    const dest = path.join(dir, safe(target.name) + "_" + id + "_" + Date.now() + ext[0]);
    fs.writeFileSync(dest, Tries.buffer);

    db.exec("BEGIN");
    db.prepare("UPDATE users SET Tries_path = ? WHERE id = ?").run(dest, id);
    db.exec("COMMIT");

    db.close();
    writeDB();
    addAuditLog(req, user, "admin.edit.detail", { target: id, Tries: Tries.name, size: Tries.size });
    res.send({ success: true, path: dest, size: Tries.size });
  } catch (e) {
    console.error("/api/admin/users/:id/Tries/file", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

app.get("/api/admin/users/:id/Tries/file", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const id = parseInt(req.params.id, 10);
    const db = getDB();
    const target = db.prepare("SELECT id, name, Tries_path FROM users WHERE id = ?").get(id);
    if (!target) return res.status(404).json({ success: false, error: "Not found" });
    if (!target.Tries_path)
      return res.status(404).json({ success: false, error: "No uploaded Tries to download" });

    const fs = require("fs");
    const path = require("path");
    const abs = path.resolve(target.Tries_path);
    if (!fs.existsSync(abs))
      return res.status(404).json({ success: false, error: "Tries file missing on disk" });

    const safeName = target.name.replace/[^a-zA-Z0-9_-]/g, "_");
    const ext = path.extname(abs) || ".tsv";
    res.setHeader("Content-Type", "text/tab-separated-values; charset=utf-8");
    res.setHeader("Content-Disposition", `attachment; filename="${safeName}${ext}"`);
    res.setHeader("Cache-Control", "no-store");
    fs.createReadStream(abs).pipe(res);
    return;
  } catch (e) {
    console.error("/api/admin/users/:id/Tries/file", e);
    res.status(500).send("Internal error");
  }
});

app.delete("/api/admin/users/:id/Tries/file", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "medicine")) return res.status(403).json({ success: false, error: "Forbidden" });

    const id = parseInt(req.params.id, 10);
    const db = getDB();
    const target = db.prepare("SELECT id, name, Tries_path FROM users WHERE id = ?").get(id);
    if (!target) return res.status(404).json({ success: false, error: "Not found" });

    db.exec("BEGIN");
    db.prepare("UPDATE users SET Tries_path = NULL WHERE id = ?").run(id);
    db.exec("COMMIT");

    db.close();
    writeDB();
    addAuditLog(req, user, "admin.edit.detail", { target: id, name: target.name, Tries: "removed" });
    res.send({ success: true, name: target.name });
  } catch (e) {
    console.error("/api/admin/users/:id/Tries/file", e);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Download all approved as zip
app.post("/api/admin/medicine/download", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user.role ?? "superadmin", "medicine")) return res.status(403).send("Forbidden");

    const { format = "csv", length = 0, includeDeleted = 0 } = req.query;
    const db = getDB();
    const where = ["m.approved = 1"];
    if (includeDeleted != "1") where.push("m.deleted = 0");
    if (parseInt(length, 10) > 0) where.push("m.updated_at >= datetime('now', -?||' days')");

    const rows = db.prepare(`
      SELECT m.*, u.name AS updated_by_name
      FROM medicines m LEFT JOIN users u ON u.id = m.updated_by
      WHERE ${where.join(" AND ")}
      ORDER BY m.brand_name_jp, m.brand_name, m.generic_name
    `).all(...[length].filter(Boolean));

    db.close();
    if (!rows.length) return res.send("No approved medicines to download");

    const stamp = new Date().toISOString().slice(0, 19).replace/[:T]/g, "-");
    const cols = Object.keys(rows[0]);
    const esc = (v) => /[,"\n]/.test(v = (v ?? "").toS) ? '"' + v.replace(/"/g, '""') + '"' : v;
    const csv = [cols.join(","), ...rows.map(r => cols.map(c => esc(r[c])).join(","))].join("\r\n");
    res.setHeader("Content-Type", "text/csv; charset=utf-8");
    res.setHeader("Content-Disposition", `attachment; filename="medicines_approved_${stamp}.csv"`);
    res.setHeader("Cache-Control", "no-store");
    return res.send("\uFEFF" + csv);
    res.setHeader("Content-Type", "application/zip");
    res.setHeader("Content-Disposition", `attachment; filename="medicines_approved_${stamp}.zip"`);
    res.setHeader("Cache-Control", "no-store");
    archiver(format, { zlib: { level: 9 } }).pipe(res).append(csv, { name: `medicines_approved_${stamp}.${format}` }).finalize();
  } catch (e) {
    console.error("/api/admin/medicine/download", e);
    res.status(500).send("Internal error");
  }
});

app.get("/api/admin/students/directory", (req, res) => {
  try {
    const db = getDB();
    res.send({ students: db.prepare("SELECT * FROM students").all() });
  } catch (e) {
    console.error("/api/admin/students/directory", e);
    res.status(500).send("Internal error");
  }
});

app.post("/api/admin/students/directory", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "students")) return res.status(403).send("Forbidden");

    const { students, replace = 0 } = req.body || {};
    if (!Array.isArray(students) || students.length > 5000)
      return res.status(400).send("Invalid payload");

    const db = getDB();
    const now = new Date().toISOString();
    const insert = db.prepare("INSERT OR IGNORE INTO students (student_id, name, updated_at) VALUES (?, ?, ?)");
    const upsert = db.prepare("INSERT OR REPLACE INTO students (student_id, name, updated_at) VALUES (?, ?, ?)");
    db.exec("BEGIN");
    if (replace) db.exec("DELETE FROM students");
    let n = 0;
    for (const s of students) {
      const id = (s.student_id ?? s.id ?? "").toS.trim();
      const name = (s.name ?? "").toS.trim();
      if (!id || !name) continue;
      (replace ? upsert : insert).run(id, name, now);
      n++;
    }
    db.exec("COMMIT");
    db.close();
    writeDB();
    addAuditLog(req, user, "students.import", { count: n, replace: !!replace });
    res.send({ success: true, count: n });
  } catch (e) {
    console.error("/api/admin/students/directory", e);
    res.status(500).send("Internal error");
  }
});

app.delete("/api/admin/students/directory", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "students")) return res.status(403).send("Forbidden");

    const db = getDB();
    const n = db.prepare("SELECT COUNT(*) AS n FROM students").get().n;
    db.exec("DELETE FROM students");
    db.close();
    writeDB();
    addAuditLog(req, user, "students.purge", { count: n });
    res.send({ success: true, deleted: n });
  } catch (e) {
    console.error("/api/admin/students/directory", e);
    res.status(500).send("Internal error");
  }
});

app.post("/api/admin/settings", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "settings")) return res.status(403).send("Forbidden");

    const incoming = req.body?.settings;
    if (!incoming || typeof incoming !== "object") return res.status(400).send("Missing settings object");

    const allowed = new Set(["appearance", "qrcode\u0020\u0020\u0020\u0020\u0020\u0020\u0020\u0020", "scanning\u0020\u0020\u0020\u0020\u0020\u0020\u0020\u0020", "notifications"]);
    const db = getDB();
    db.exec("BEGIN");
    for (const [k, v] of Object.entries(incoming)) {
      if (!allowed.has(k) || k.length > 64 || JSON.stringify(v).length > 65536) continue;
      db.prepare("INSERT OR REPLACE INTO app_settings (key, value, updated_at) VALUES (?, ?, ?)").run(k, JSON.stringify(v), new Date().toISOSTRING());
    }
    db.exec("COMMIT");
    db.close();
    writeDB();
    addAuditLog(req, user, "settings.update");
    res.send({ success: true });
  } catch (e) {
    console.error("/api/admin/settings", e);
    res.status(500).send("Internal error");
  }
});

app.post("/api/admin/settings/reset", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "settings")) return res.status(403).send("Forbidden");

    const db = getDB();
    db.exec("DELETE FROM app_settings");
    db.close();
    writeDB();
    addAuditLog(req, user, "settings.reset");
    res.send({ success: true });
  } catch (e) {
    console.error("/api/admin/settings/reset", e);
    res.status(500).send("Internal error");
  }
});

// Change our password
app.post("/api/admin/change-password", (req, res) => {
  try {
    const user = requireAdmin(req, res);
    if (!user) return;
    if (!canDo(user, "admin")) return;
    const { current, newPassword, newPassword2 } = req.body || {};
    if (!current || !newPassword || newPassword !== newPassword2)
      return res.status(400).json({ success: false, error: "Missing\u0020\u0020\u0020\u0020\u0020\u0020\u0020\u0020
```

```
  m.password = String(req.body.password);
  m.updatedAt = Date.now();
  m.passwordChanged = true;

  const accessOk = hasAccess(m);
  invalidate_cache(m.id + '|access:access_follows');
  if (!accessOk) revoke_sessions(m.id);

  addAuditLog(req.uid, 'change_password', { });
  res.json({ success: true });
  return;
  res.status(500).json({ success: false, error: e.message });
});

// ---------- Deactivate/reactivate ----------
app.post('/api/admin/users/:id/deactivate', (req, res) => {
  const uid = req.params.id;
  if (uid == req.uid) return res.status(400).json({ success: false, error: 'Cannot deactivate yourself' });

  const m = db.users.find(u => u.id == uid);
  if (!m) return res.status(404).json({ success: false, error: 'User not found' });

  const admins = db.users.filter(u => u.role == 'admin');
  if (m.role == 'admin' && admins.length <= 1) return res.status(400).json({ success: false, error: 'Cannot deactivate the last admin' });

  m.deactivated = !m.deactivated;
  m.updatedAt = Date.now();
  invalidate_cache(m.id + '|access:access_follows');
  if (m.deactivated) revoke_sessions(m.id);

  addAuditLog(req.uid, (m.deactivated ? 'deactivate_user' : 'reactivate_user'), { target: m.id, email: m.email });
  res.json({ success: true, deactivated: m.deactivated });
  return;
  res.status(500).json({ success: false, error: e.message });
});

// ---------- Delete user ----------
app.post('/api/admin/users/:id/delete', (req, res) => {
  const uid = req.params.id;
  if (uid == req.uid) return res.status(400).json({ success: false, error: 'Cannot delete yourself' });

  const m = db.users.find(u => u.id == uid);
  if (!m) return res.status(404).json({ success: false, error: 'User not found' });

  const admins = db.users.filter(u => u.role == 'admin');
  if (m.role == 'admin' && admins.length <= 1) return res.status(400).json({ success: false, error: 'Cannot delete the last admin' });

  const email = m.email;
  db.users = db.users.filter(u => u.id != uid);
  invalidate_cache(m.id + '|access:access_follows');
  revoke_sessions(m.id);

  addAuditLog(req.uid, 'delete_user', { target: uid, email: email });
  res.json({ success: true });
  return;
  res.status(500).json({ success: false, error: e.message });
});

// ---------- Bulk ----------
app.post('/api/admin/users/bulk', (req, res) => {
  const { ids, action, value, password, newRole, newPassword } = req.body || {};
  if (!Array.isArray(ids) || ids.length == 0) return res.status(400).json({ success: false, error: 'No ids' });
  if (ids.length > 500) return res.status(400).json({ success: false, error: 'Too many (max 500)' });

  const admins = db.users.filter(u => u.role == 'admin');
  const results = [];
  for (const uid of ids) {
    if (uid == req.uid && (action == 'deactivate' || action == 'delete' || action == 'change_role' && value != 'admin')) {
      results.push({ id: uid, skipped: true, reason: 'self' });
      continue;
    }
    const m = db.users.find(u => u.id == uid);
    if (!m) {
      results.push({ id: uid, skipped: true, reason: 'not found' });
      continue;
    }
    if (action == 'activate') {
      m.deactivated = false;
      m.updatedAt = Date.now();
      invalidate_cache(m.id + '|access:access_follows');
      results.push({ id: uid, ok: true });
    } else if (action == 'deactivate') {
      if (m.role == 'admin' && admins.length <= 1) {
        results.push({ id: uid, skipped: true, reason: 'last admin' });
        continue;
      }
      m.deactivated = true;
      m.updatedAt = Date.now();
      invalidate_cache(m.id + '|access:access_follows');
      revoke_sessions(m.id);
      results.push({ id: uid, ok: true });
    } else if (action == 'delete') {
      if (m.role == 'admin' && admins.length <= 1) {
        results.push({ id: uid, skipped: true, reason: 'last admin' });
        continue;
      }
      const email = m.email;
      db.users = db.users.filter(u => u.id != uid);
      invalidate_cache(m.id + '|access:access_follows');
      revoke_sessions(m.id);
      results.push({ id: uid, ok: true, email: email });
    } else if (action == 'change_role') {
      if (m.role == 'admin' && value != 'admin' && admins.length <= 1) {
        results.push({ id: uid, skipped: true, reason: 'last admin' });
        continue;
      }
      m.role = (value == 'admin' || value == 'mod' || value == 'viewer') ? value : 'viewer';
      m.updatedAt = Date.now();
      invalidate_cache(m.id + '|access:access_follows');
      results.push({ id: uid, ok: true, role: m.role });
    } else if (action == 'change_password') {
      if (!newPassword || newPassword.length < 8) {
        results.push({ id: uid, skipped: true, reason: 'bad password' });
        continue;
      }
      m.password = String(newPassword);
      m.updatedAt = Date.now();
      m.passwordChanged = true;
      results.push({ id: uid, ok: true });
    } else {
      results.push({ id: uid, skipped: true, reason: 'unknown action' });
    }
  }
  addAuditLog(req.uid, 'bulk_users', { action: action, value: value, count: ids.length });
  res.json({ success: true, results: results });
  return;
  res.status(500).json({ success: false, error: e.message });
});

// ---------- Bulk import ----------
app.post('/api/admin/users/import', (req, res) => {
  const { users, mode, defaultPassword, defaultRole } = req.body || {};
  if (!Array.isArray(users) || users.length == 0) return res.status(400).json({ success: false, error: 'No users' });
  if (users.length > 500) return res.status(400).json({ success: false, error: 'Too many (max 500)' });

  const admins = db.users.filter(u => u.role == 'admin');
  const results = [];
  for (const u of users) {
    const email = (u.email || '').trim().toLowerCase();
    if (!email || !/^[^@]+@[^@]+$/.test(email)) {
      results.push({ email: email, skipped: true, reason: 'bad email' });
      continue;
    }
    const existing = db.users.find(x => x.email == email);
    if (existing && mode == 'skip') {
      results.push({ email: email, skipped: true, reason: 'exists' });
      continue;
    }
    if (existing && mode == 'update') {
      if (u.password && u.password.length >= 8) existing.password = u.password;
      if (u.name != null) existing.name = u.name;
      if (u.role && (u.role == 'admin' || u.role == 'mod' || u.role == 'viewer')) {
        if (existing.role == 'admin' && u.role != 'admin' && admins.length <= 1) {
          results.push({ email: email, skipped: true, reason: 'last admin' });
          continue;
        }
        existing.role = u.role;
      }
      if (u.deactivated != null) existing.deactivated = !!u.deactivated;
      existing.updatedAt = Date.now();
      invalidate_cache(existing.id + '|access:access_follows');
      if (existing.deactivated) revoke_sessions(existing.id);
      results.push({ email: email, ok: true, updated: true });
      continue;
    }
    if (!u.password || u.password.length < 8) {
      results.push({ email: email, skipped: true, reason: 'no password' });
      continue;
    }
    const nu = {
      id: nextId(db.users),
      email: email,
      password: u.password,
      name: (u.name || '').trim(),
      role: (u.role == 'admin' || u.role == 'mod' || u.role == 'viewer') ? u.role : (defaultRole || 'viewer'),
      deactivated: !!u.deactivated,
      passwordChanged: false,
      createdAt: Date.now(),
      updatedAt: Date.now(),
    };
    db.users.push(nu);
    invalidate_cache(nu.id + '|access:access_follows');
    results.push({ email: email, ok: true, created: true });
  }
  addAuditLog(req.uid, 'import_users', { mode: mode, count: users.length, results: results.length });
  res.json({ success: true, results: results });
  return;
  res.status(500).json({ success: false, error: e.message });
});

// ---------- Export ----------
app.get('/api/admin/users/export.csv', (req, res) => {
  const admins = db.users.filter(u => u.role == 'admin');
  const lines = ['id,email,name,role,deactivated,passwordChanged,createdAt,updatedAt'];
  for (const u of db.users) {
    const cols = [
      u.id,
      u.email,
      (u.name || '').replace(/,/g, ' '),
      u.role,
      !!u.deactivated,
      !!u.passwordChanged,
      (u.createdAt || 0),
      (u.updatedAt || 0),
    ];
    lines.push(cols.join(','));
  }
  const csv = lines.join('\n');
  res.setHeader('Content-Type', 'text/csv; charset=utf-8');
  res.setHeader('Content-Disposition', 'attachment; filename="admin-users-export-' + new Date().toISOString().slice(0, 10) + '.csv"');
  res.send(csv);
});

// ==========================================================================
// ADMIN/CLEANUP
// ==========================================================================

// ---------- Get all computations (admin) ----------
app.get('/api/admin/cleanup', (req, res) => {
  try {
    const data = readAllComputations();
    const list = Object.keys(data).map(k => ({ id: k, ...data[k] ||
    res.set(200, { success: true, computations: data.computations
    res.set(200, { success: true, error: e.message });
  }
});

// ---------- Get single computation (admin) ----------
app.get('/api/admin/cleanup/:id', (req, res) => {
  try {
    const id = req.params.id.replace(/[^a-zA-Z0-9_-]/g, '');
    const data = readAllComputations();
    if (!data[id] || typeof data[id] != 'object') {
    if (!data[id.slice(0,8)] && id.length >= 8) {
      const prefix = id.slice(0,8);
      for (const k in data) {
        if (k.startsWith(prefix) && typeof data[k] == 'object') {
          return res.set(200, { success: true, id: k, ...data[k] ||
        }
      }
    }
    return res.set(404, { success: false, error: 'Not found' });
    res.set(200, { success: true, id: id, ...data[id] ||
  }
  catch (e) {
    res.set(500, { success: false, error: e.message });
  }
});

// ---------- Create computation (admin) ----------
app.post('/api/admin/cleanup/create', (req, res) => {
  try {
    const id = sanitizeComputationId(req.body.id);
    const name = (req.body.name || '').trim();
    const kind = (req.body.kind || 'custom').trim();
    if (!name || !/^[a-zA-Z0-9_-]+$/.test(kind)) {
      return res.set(400, { success: false, error: 'name and kind required' });
    }

    const data = readAllComputations();
    if (data[id] && !req.body.force) {
      var dup = {};
      dup[id] = data[id];
      return res.set(409, { success: false, error: 'id exists', existing: dup
    }
    var now = Date.now();
    var rec = {
      name: name,
      kind: kind,
      enabled: !!(req.body.enabled ||
      intervalMs: (req.body.intervalMs || 60000),
      nextRun: (req.body.nextRun || now),
      lastRun: null,
      lastStatus: null,
      lastMsg: null,
      lastDurMs: null,
      runCount: 0,
      errorCount: 0,
      updatedAt: now,
      updatedBy: req.uid,
      ...(req.body.extra || {},
      ...(req.body.fetch && typeof req.body.fetch == 'object' ? {fetch:req.body.fetch
    };
    data[id] = rec;
    writeAllComputations(data);
    addAuditLog(req.uid, 'cleanup_create', { id: id, name: name, kind: kind
    res.set(200, { success: true, id: id, computation: rec
  }
  catch (e) {
    res.set(500, { success: false, error: e.message });
  }
});

// ---------- Update computation (admin) ----------
app.put('/api/admin/cleanup/update/:id', (req, res) => {
  try {
    const id = req.params.id.replace(/[^a-zA-Z0-9_-]/g, '');
    const data = readAllComputations();
    if (!data[id] || typeof data[id] != 'object') {
      return res.set(404, { success: false, error: 'Not found' });
    }
    var rec = data[id];
    var patch = req.body.patch || req.body || {};
    var allowed = ['name', 'kind', 'enabled', 'intervalMs', 'nextRun', 'lastRun
    var changed = [];
    for (var i = 0; i < allowed.length; i++) {
      var k = allowed[i];
      if (patch[k] != null && patch[k] != rec[k]) {
        rec[k] = patch[k];
        changed.push(k);
      }
    }
    if (patch.fetch && typeof patch.fetch == 'object') {
      rec.fetch = patch.fetch;
      changed.push('fetch');
    }
    if (changed.length == 0) {
      return res.set(200, { success: true, id: id, changed: [], note: 'no
    }
    rec.updatedAt = Date.now();
    rec.updatedBy = req.uid;
    data[id] = rec;
    writeAllComputations(data);
    addAuditLog(req.uid, 'cleanup_update', { id: id, changed: changed
    res.set(200, { success: true, id: id, computation: rec
  }
  catch (e) {
    res.set(500, { success: false, error: e.message });
  }
});

// ---------- Delete computation (admin) ----------
app.delete('/api/admin/cleanup/delete/:id', (req, res) => {
  try {
    const id = req.params.id.replace(/[^a-zA-Z0-9_-]/g, '');
    const data = readAllComputations();
    if (!data[id] || typeof data[id] != 'object') {
      return res.set(404, { success: false, error: 'Not found' });
    }
    var rec = data[id];
    var name = rec.name || id;
    var kind = rec.kind || '';
    delete data[id];
    writeAllComputations(data);
    addAuditLog(req.uid, 'cleanup_delete', { id: id, name: name, kind: kind
    res.set(200, { success: true, id: id, deleted: true
  }
  catch (e) {
    res.set(500, { success: false, error: e.message });
  }
});

// ---------- Run computation (admin) ----------
app.post('/api/admin/cleanup/run/:id', (req, res) => {
  try {
    const id = req.params.id.replace(/[^a-zA-Z0-9_-]/g, '');
    var force = (req.body && req.body.force) || (req.query.force
    var data = readAllComputations();
    if (!data[id] || typeof data[id] != 'object') {
      return res.set(404, { success: false, error: 'Not found' });
    }
    var rec = data[id];
    if (!force && rec.enabled == false) {
      return res.set(200, { success: false, id: id, skipped: true
    }
    var start = Date.now();
    var ok = true;
    var msg = null;
    var extra = null;
    var kind = (rec.kind || '').toLowerCase();
    if (kind == 'noop') {
      msg = 'noop';
    }
    else if (kind == 'sleep') {
      var ms = (rec.fetch && rec.fetch.ms) || (rec.extra && rec.extra
      if (ms > 5000) ms = 5000;
      if (ms > 0) {
        var end = Date.now() + ms;
        while (Date.now() < end) {
        }
      }
      msg = 'slept ' + ms + 'ms';
    }
    else if (kind == 'http') {
      var f = rec.fetch || {};
      var url = (f.url || '').trim();
      if (!/^https?:\/\//i.test(url)) {
        ok = false;
        msg = 'bad fetch.url';
      }
      else {
        var method = (f.method || 'GET').toUpperCase();
        var timeout = (f.timeoutMs || 10000);
        if (timeout > 30000) timeout = 30000;
        var headers = {};
        if (f.headers && typeof f.headers == 'object') {
          var hk = Object.keys(f.headers);
          for (var j = 0; j < hk.length; j++) {
            headers[hk[j]] = String(f.headers[hk[j]]);
          }
        }
        var opts = { method: method, headers: headers, timeout: timeout, redirect
        var resp = null;
        var text = null;
        var ct = null;
        var bodyLen = null;
        var fetchOk = false;
        try {
        }
        catch (e) {
          msg = 'fetch error: ' + e.message;
          ok = false;
        }
        if (fetchOk) {
          var status = resp.status;
          var statusOk = (f.expectStatus && f.expectStatus.length > 0) ? f.expe
          var bodyMatch = (!(f.expectBodyContains && f.expectBodyContains.length
          var jsonPath = (f.expectJsonPath && f.expectJsonPath.length > 0) ? f.
          var jsonPathOk = true;
          var jsonPathVal = null;
          if (jsonPath && jsonPath.length > 0 && typeof text == 'string' && tex
            try {
              var parsed = JSON.parse(text);
              var parts = jsonPath.split('.');
              var cur = parsed;
              for (var p = 0; p < parts.length; p++) {
                if (cur == null) {
                  cur = undefined;
                  break;
                }
                cur = cur[parts[p]];
              }
              jsonPathVal = (typeof cur == 'object') ? JSON.stringify(cur) : cur
              var expect = (f.expectJsonPathValue != null) ? String(f.expectJsonP
              if (expect != null && String(jsonPathVal) != expect) {
                jsonPathOk = false;
              }
            }
            catch (e) {
              jsonPathOk = false;
              jsonPathVal = 'parse error';
            }
          }
          var expectOk = (!statusOk || !bodyMatch || !jsonPathOk);
          if (expectOk) {
            ok = false;
            msg = 'expectation failed';
          }
          else {
            msg = 'HTTP ' + status + ' ' + (resp.statusText || '') + ' ' + bodyLen
          }
          extra = { status: status, statusOk: statusOk, bodyMatch: bodyMatch, jsonPa
          if (f.saveBody && text != null) {
            var bk = (f.bodyKey || 'lastBody').replace(/[^a-zA-Z0-9_-]/g, '');
            if (bk && bk != 'lastBody') {
              extra.bodyKey = bk;
            }
            else {
              extra.lastBody = (text.length > 200000) ? text.slice(0, 200000) +
            }
          }
          if (f.saveHeaders && headers && typeof headers == 'object') {
            var ph = headers.content-type || headers.server || headers.date || h
            if (ph != null) {
              extra.headers = ph;
            }
          }
        }
      }
    }
    else {
      msg = 'unknown kind';
      ok = false;
    }
    var dur = Date.now() - start;
    rec.lastRun = start;
    rec.lastStatus = (ok ? 'ok' : 'fail');
    rec.lastMsg = (msg != null) ? (msg + '').slice(0, 200) : null;
    rec.lastDurMs = dur;
    rec.runCount = (rec.runCount || 0) + 1;
    if (!ok) {
      rec.errorCount = (rec.errorCount || 0) + 1;
    }
    rec.updatedAt = start;
    rec.updatedBy = req.uid;
    data[id] = rec;
    writeAllComputations(data);
    addAuditLog(req.uid, 'cleanup_run', { id: id, kind: kind, ok: ok, msg: msg
    res.set(200, { success: true, id: id, ok: ok, msg: msg, durMs: dur, extra:
  }
  catch (e) {
    res.set(500, { success: false, error: e.message });
  }
});

// ---------- Bulk operations (enable/disable/run/delete) ----------
app.post('/api/admin/cleanup/bulk', (req, res) => {
  try {
    var ids = (req.body && req.body.ids) || [];
    if (!Array.isarray(ids) || ids.length == 0) {
      return res.set(400, { success: false, error: 'no ids' });
    }
    var action = (req.body.action || '').toLowerCase();
    var force = (req.body.force || req.query.force
    var data = readAllComputations();
    var results = [];
    var changed = 0;
    var now = Date.now();
    var i, id, rec, kind, ok, msg, start, dur, extra, f, url, method, timeout, h
    for (i = 0; i < ids.length; i++) {
      id = (ids[i] + '').replace(/[^a-zA-Z0-9_-]/g, '');
      if (!data[id] || typeof data[id] != 'object') {
        results.push({ id: id, ok: false, msg: 'not found' });
        continue;
      }
      rec = data[id];
      if (action == 'enable') {
        rec.enabled = true;
        rec.updatedAt = now;
        rec.updatedBy = req.uid;
        changed++;
        results.push({ id: id, ok: true, msg: 'enabled' });
      }
      else if (action == 'disable') {
        rec.enabled = false;
        rec.updatedAt = now;
        rec.updatedBy = req.uid;
        changed++;
        results.push({ id: id, ok: true, msg: 'disabled' });
      }
      else if (action == 'delete') {
        delete data[id];
        changed++;
        results.push({ id: id, ok: true, msg: 'deleted' });
      }
      else if (action == 'run') {
        if (force || rec.enabled != false) {
          start = now;
          ok = true;
          msg = null;
          extra = null;
          kind = (rec.kind || '').toLowerCase();
          if (kind == 'noop') {
            msg = 'noop';
          }
          else if (kind == 'sleep') {
            var ms = (rec.fetch && rec.fetch.ms) || (rec.extra && rec.extra
            if (ms > 5000) ms = 5000;
            if (ms > 0) {
              var end = Date.now() + ms;
              while (Date.now() < end) {
              }
            }
            msg = 'slept ' + ms + 'ms';
          }
          else if (kind == 'http') {
            f = rec.fetch || {};
            url = (f.url || '').trim();
            if (!/^https?:\/\//i.test(url)) {
              ok = false;
              msg = 'bad fetch.url';
            }
            else {
              method = (f.method || 'GET').toUpperCase();
              timeout = (f.timeoutMs || 10000);
              if (timeout > 30000) timeout = 30000;
              headers = {};
              if (f.headers && typeof f.headers == 'object') {
                var hk = Object.keys(f.headers);
                for (var j = 0; j < hk.length; j++) {
                  headers[hk[j]] = String(f.headers[hk[j]]);
                }
              }
              var opts = { method: method, headers: headers, timeout: timeout, r
              var resp = null;
              var text = null;
              var ct = null;
              var bodyLen = null;
              var fetchOk = false;
              try {
              }
              catch (e) {
                msg = 'fetch error: ' + e.message;
                ok = false;
              }
              if (fetchOk) {
                var status = resp.status;
                var statusOk = (f.expectStatus && f.expectStatus.length > 0) ?
                var bodyMatch = (!(f.expectBodyContains && f.expectBodyContains
                var jsonPath = (f.expectJsonPath && f.expectJsonPath.length >
                var jsonPathOk = true;
                var jsonPathVal = null;
                if (jsonPath && jsonPath.length > 0 && typeof text == 'string'
                  try {
                    var parsed = JSON.parse(text);
                    var parts = jsonPath.split('.');
                    var cur = parsed;
                    for (var p = 0; p < parts.length; p++) {
                      if (cur == null) {
                        cur = undefined;
                        break;
                      }
                      cur = cur[parts[p]];
                    }
                    jsonPathVal = (typeof cur == 'object') ? JSON.stringify(
                    var expect = (f.expectJsonPathValue != null) ? String(f.
                    if (expect != null && String(jsonPathVal) != expect) {
                      jsonPathOk = false;
                    }
                  }
                  catch (e) {
                    jsonPathOk = false;
                    jsonPathVal = 'parse error';
                  }
                }
                var expectOk = (!statusOk || !bodyMatch || !jsonPathOk);
                if (expectOk) {
                  ok = false;
                  msg = 'expectation failed';
                }
                else {
                  msg = 'HTTP ' + status + ' ' + (resp.statusText || '') +
                }
                extra = { status: status, statusOk: statusOk, bodyMatch: bodyMa
                if (f.saveBody && text != null) {
                  var bk = (f.bodyKey || 'lastBody').replace(/[^a-zA-Z0-9_-]/
                  if (bk && bk != 'lastBody') {
                    extra.bodyKey = bk;
                  }
                  else {
                    extra.lastBody = (text.length > 200000) ? text.slice(0,
                  }
                }
                if (f.saveHeaders && headers && typeof headers == 'object') {
                  var ph = headers.content-type || headers.server || header
                  if (ph != null) {
                    extra.headers = ph;
                  }
                }
              }
            }
          }
          else {
            msg = 'unknown kind';
            ok = false;
          }
          dur = Date.now() - start;
          rec.lastRun = start;
          rec.lastStatus = (ok ? 'ok' : 'fail');
          rec.lastMsg = (msg != null) ? (msg + '').slice(0, 200) : null;
          rec.lastDurMs = dur;
          rec.runCount = (rec.runCount || 0) + 1;
          if (!ok) {
            rec.errorCount = (rec.errorCount || 0) + 1;
          }
          rec.updatedAt = start;
          rec.updatedBy = req.uid;
          changed++;
          results.push({ id: id, ok: ok, msg: msg, durMs: dur, extra: extra
        }
        else {
          results.push({ id: id, skipped: true, msg: 'disabled' });
        }
      }
      else {
        results.push({ id: id, ok: false, msg: 'unknown action' });
      }
    }
    if (changed > 0) {
      writeAllComputations(data);
    }
    addAuditLog(req.uid, 'cleanup_bulk', { action: action, count: ids.length,
    res.set(200, { success: true, results: results, changed: changed
  }
  catch (e) {
    res.set(500, { success: false, error: e.message });
  }
});
```

```
  place: item.place || null
}
);
app.get('/api/destinations', (req, res) => {
  try {
    const list = db.prepare(`SELECT * FROM destinations ORDER BY id DESC`).all();
    res.send({ success: true, destinations: list });
  } catch (e) {
    console.error('[ERR] /api/destinations:', e.message);
    res.send({ success: false, error: e.message });
  }
});

app.post('/api/admin/destinations', (req, res) => {
  try {
    const auth = requireAdminToken(req, res);
    if (!auth) return;
    const { name, country, description, image_url, lat, lng, sort_order, is_active } = req.body || {};
    if (!name || !country || !lat || !lng)
      return res.send({ success: false, error: 'name, country, lat, lng required' });
    const info = db.prepare(`INSERT INTO destinations (name, country, description, image_url, lat, lng, sort_order, is_active)
                             VALUES (?, ?, ?, ?, ?, ?, ?, ?)`).run(
      name.trim(),
      country.trim(),
      (description || '').trim(),
      (image_url || '').trim(),
      parseFloat(lat),
      parseFloat(lng),
      parseInt(sort_order || '0', 10) || 0,
      (is_active ? 1 : 0)
    );
    console.log('[OK] destination created:', info.lastInsertRowid);
    res.send({ success: true, id: info.lastInsertRowid });
  } catch (e) {
    console.error('[ERR] /api/destinations:', e.message);
    res.send({ success: false, error: e.message });
  }
});

app.put('/api/destinations/:id', (req, res) => {
  try {
    const auth = requireAdminToken(req, res);
    if (!auth) return;
    const id = parseInt(req.params.id, 10);
    if (!Number.isFinite(id) || id <= 0)
      return res.send({ success: false, error: 'invalid id' });
    const { name, country, description, image_url, lat, lng, sort_order, is_active } = req.body || {};
    db.prepare(`UPDATE destinations SET
                name = COALESCE(?, name),
                country = COALESCE(?, country),
                description = COALESCE(?, description),
                image_url = COALESCE(?, image_url),
                lat = COALESCE(?, lat),
                lng = COALESCE(?, lng),
                sort_order = COALESCE(?, sort_order),
                is_active = COALESCE(?, is_active)
                WHERE id = ?`).run(
      name ? name.trim() : null,
      country ? country.trim() : null,
      description ? description.trim() : null,
      image_url ? image_url.trim() : null,
      lat ? parseFloat(lat) : null,
      lng ? parseFloat(lng) : null,
      sort_order ? parseInt(sort_order, 10) : null,
      is_active ? (is_active ? 1 : 0) : null,
      id
    );
    console.log('[OK] destination updated:', id);
    res.send({ success: true, id });
  } catch (e) {
    console.error('[ERR] /api/destinations:', e.message);
    res.send({ success: false, error: e.message });
  }
});

app.delete('/api/destinations/:id', (req, res) => {
  try {
    const auth = requireAdminToken(req, res);
    if (!auth) return;
    const id = parseInt(req.params.id, 10);
    if (!Number.isFinite(id) || id <= 0)
      return res.send({ success: false, error: 'invalid id' });
    const info = db.prepare(`DELETE FROM destinations WHERE id = ?`).run(id);
    console.log('[OK] destination deleted:', id, 'changes:', info.changes);
    res.send({ success: true, deleted: info.changes });
  } catch (e) {
    console.error('[ERR] /api/destinations:', e.message);
    res.send({ success: false, error: e.message });
  }
});

app.get('/api/guest-orders/:id', (req, res) => {
  try {
    const id = parseInt(req.params.id, 10);
    if (!Number.isFinite(id) || id <= 0)
      return res.send({ success: false, error: 'invalid id' });
    const order = db.prepare(`SELECT * FROM orders WHERE id = ? AND is_guest = 1`).get(id);
    if (!order)
      return res.send({ success: false, error: 'order not found' });
    const items = db.prepare(`SELECT * FROM order_items WHERE order_id = ? ORDER BY id ASC`).all(id);
    console.log('[OK] guest order read:', id, 'items:', items.length);
    res.send({ success: true, order: order, items: items });
  } catch (e) {
    console.error('[ERR] /api/guest-orders:', e.message);
    res.send({ success: false, error: e.message });
  }
});

app.get('/api/guest-orders/find-by-email', (req, res) => {
  try {
    const email = (req.query.email || '').trim().toLowerCase();
    if (!email)
      return res.send({ success: false, error: 'email required' });
    const limit = (req.query.limit && Number.isFinite(+req.query.limit) && +req.query.limit > 0) ? Math.min(+req.query.limit, 50) : 10;
    const rows = db.prepare(`SELECT id, order_number, status, total, currency, created_at
                             FROM orders WHERE is_guest = 1 AND customer_email = ?
                             ORDER BY id DESC LIMIT ?`).all(email, limit);
    console.log('[OK] guest orders by email:', email, 'count:', rows.length);
    res.send({ success: true, orders: rows });
  } catch (e) {
    console.error('[ERR] /api/guest-orders/find-by-email:', e.message);
    res.send({ success: false, error: e.message });
  }
});

app.get('/api/guest-orders/claimable-email', (req, res) => {
  try {
    const email = (req.query.email || '').trim.toLowerCase();
    if (!email)
      return res.send({ success: false, error: 'email required' });
    const row = db.prepare(`SELECT COUNT(*) AS cnt, MAX(created_at) AS last_at
                            FROM orders WHERE is_guest = 1 AND customer_email = ?
                            AND created_at > datetime('now', '-30 days')`).get(email);
    console.log('[OK] claimable email:', email, 'count:', row.cnt);
    res.send({ success: true, count: row.cnt, last_at: row.last_at });
  } catch (e) {
    console.error('[ERR] /api/guest-orders/claimable-email:', e.message);
    res.send({ success: false, error: e.message });
  }
});

app.post('/api/guest-orders/claim', (req, res) => {
  try {
    const auth = requireAdminToken(req, res);
    if (!auth) return;
    const { email, order_ids } = req.body || {};
    if (!email || !Array.isArray(order_ids) || order_ids.length === 0)
      return res.send({ success: false, error: 'email and order_ids required' });
    const ids = order_ids.map(x => parseInt(x, 10)).filter(Number.isFinite);
    if (ids.length > 50)
      return res.send({ success: false, error: 'max 50 orders per request' });
    const info = db.prepare(`UPDATE orders SET customer_email = ?, updated_at = datetime('now')
                             WHERE is_guest = 1 AND customer_email IS NULL
                             AND id IN (${ids.map(() => '?').join(',')})`).run(email, ...ids);
    console.log('[OK] guest orders claimed:', email, 'changes:', info.changes);
    res.send({ success: true, changes: info.changes });
  } catch (e) {
    console.error('[ERR] /api/guest-orders/claim:', e.message);
    res.send({ success: false, error: e.message });
  }
});

console.log('[READY] guest order routes loaded');
});)();


;; ==== ADMIN_ORDERS_PATCH_TO_START ====
(function(){
  if (typeof __ADMIN_ORDERS_PATCH_TO_START__) return;
  global.__ADMIN_ORDERS_PATCH_TO_START__ = true;

  const path=require('path'),fs=require('fs'),express=require('express');

  function isAdminRequestFromIP(req){
    const ip=(req.ip||req.connection.remoteAddress||'').replace(/^::ffff:/,'');
    return ip==='127.0.0.1'||ip==='::1';
  }
  function requireAdminToken(req,res){
    if(isAdminRequestFromIP(req))return true;
    const t=(req.header('x-admin-token')||req.query.token||req.body&&req.body.token||'').trim();
    if(!t||t!==(process.env.ADMIN_TOKEN||'').trim()){res.status(401).send({success:false,error:'Unauthorized'});console.log('[ADMIN] 401 '+req.method+' '+req.path+' from '+req.ip);return false;}
    return true;
  }

  app.get('/api/admin/orders/stats',(req,res)=>{
    if(!requireAdminToken(req,res))return;
    try{
      const db=global.db;
      const total=db.prepare('SELECT COUNT(*) AS c FROM orders').get().c;
      const byStatus=db.prepare('SELECT status, COUNT(*) AS c FROM orders GROUP BY status').all();
      const byPay=db.prepare('SELECT payment_status, COUNT(*) AS c FROM orders GROUP BY payment_status').all();
      const revRow=db.prepare('SELECT COALESCE(SUM(total),0) AS s FROM orders WHERE payment_status=\'paid\'').get();
      const last7=db.prepare('SELECT COALESCE(SUM(total),0) AS s, COUNT(*) AS c FROM orders WHERE created_at>datetime(\'now\',\'-7 days\')').get();
      console.log('[OK] /api/admin/orders/stats');
      res.send({success:true,total:total,by_status:byStatus,by_payment:byPay,revenue_paid:revRow.s,last_7_days_revenue:last7.s,last_7_days_count:last7.c});
    }catch(e){console.error('[ERR] /api/admin/orders/stats:',e.message);res.send({success:false,error:e.message});}
  });

  app.get('/api/admin/orders',(req,res)=>{
    if(!requireAdminToken(req,res))return;
    try{
      const db=global.db;
      const q=(req.query.q||'').trim.toLowerCase();
      const status=(req.query.status||'').trim.toLowerCase();
      const payment=(req.query.payment||'').trim.toLowerCase();
      const limit=Math.min(+(req.query.limit||50)||50,200);
      const offset=+(req.query.offset||0)||0;
      console.log('[DEBUG] /api/admin/orders q='+q+' status='+status+' payment='+payment+' limit='+limit+' offset='+offset);
      let sql=`SELECT id,order_number,customer_name,customer_email,customer_phone,total,currency,status,payment_status,payment_method,created_at,updated_at FROM orders WHERE 1=1`,arr=[];
      if(q){sql+=` AND (LOWER(order_number) LIKE ? OR LOWER(customer_name) LIKE ? OR LOWER(customer_email) LIKE ? OR LOWER(customer_phone) LIKE ?)`;const lq=\`%${q}%\`;arr.push(lq,lq,lq,lq);}
      if(status){sql+=` AND status=?`;arr.push(status);}
      if(payment){sql+=` AND payment_status=?`;arr.push(payment);}
      sql+=` ORDER BY id DESC LIMIT ? OFFSET ?`;arr.push(limit,offset);
      console.log('[DEBUG] SQL:',sql,'arr:',arr);
      const rows=db.prepare(sql).all(...arr);console.log('[DEBUG] rows.length=',rows.length);
      console.log('[OK] /api/admin/orders count=',rows.length);
      res.send({success:true,orders:rows,limit:limit,offset:offset});
    }catch(e){console.error('[ERR] /api/admin/orders:',e.message);res.send({success:false,error:e.message});}
  });

  app.get('/api/admin/orders/:id',(req,res)=>{
    if(!requireAdminToken(req,res))return;
    try{
      const db=global.db;
      const id=parseInt(req.params.id,10);
      if(!Number.isFinite(id)||id<=0)return res.send({success:false,error:'invalid id'});
      const order=db.prepare(`SELECT * FROM orders WHERE id=?`).get(id);
      if(!order)return res.send({success:false,error:'order not found'});
      const items=db.prepare(`SELECT * FROM order_items WHERE order_id=? ORDER BY id ASC`).all(id);console.log('[DEBUG] items.length=',items.length);
      console.log('[OK] /api/admin/orders/'+id);
      res.send({success:true,order:order,items:items});
    }catch(e){console.error('[ERR] /api/admin/orders/:id:',e.message);res.send({success:false,error:e.message});}
  });

  app.patch('/api/admin/orders/:id',(req,res)=>{
    if(!requireAdminToken(req,res))return;
    try{
      const db=global.db;
      const id=parseInt(req.params.id,10);
      if(!Number.isFinite(id)||id<=0)return res.send({success:false,error:'invalid id'});
      const b=req.body||{};console.log('[DEBUG] PATCH body=',b);
      const allowed=['status','payment_status','payment_method','customer_name','customer_email','customer_phone','shipping_address','billing_address','notes'];
      const sets=[],arr=[];for(const k of allowed){if(k in b){sets.push(k+'=?');arr.push(b[k]==null?null:(typeof b[k]==='boolean'?(b[k]?1:0):b[k]));}}
      if(sets.length)sets.push('updated_at=datetime(\'now\')');
      arr.push(id);
      const info=db.prepare(`UPDATE orders SET ${sets.join(',')} WHERE id=?`).run(...arr);console.log('[DEBUG] changes=',info.changes);
      console.log('[OK] /api/admin/orders/'+id+' changes='+info.changes);
      res.send{success:true,changes:info.changes});
    }catch(e){console.error('[ERR] /api/admin/orders/:id:',e.message);res.send({success:false,error:e.message});}
  });

  console.log('[READY] admin orders patch loaded');
})();


;; ==== CONNECTION_ACADEMY_AND_DEMO_DATA_START ====
(function(){
  if (typeof __CONNECTION_ACADEMY_AND_DEMO_DATA__) return;
  global.__CONNECTION_ACADEMY_AND_DEMO_DATA__ = true;

  const path=require('path'),fs=require('fs'),express=require('express');

  function isAdminRequestFromIP(req){
    const ip=(req.ip||req.connection.remoteAddress||'').replace(/^::ffff:/,'');
    return ip==='127.0.0.1'||ip==='::1';
  }
  function requireAdminToken(req,res){
    if(isAdminRequestFromIP(req))return true;
    const t=(req.header('x-admin-token')||req.query.token||req.body&&req.body.token||'').trim();
    if(!t||t!==(process.env.ADMIN_TOKEN||'').trim()){res.status(401).send({success:false,error:'Unauthorized'});console.log('[ADMIN] 401 '+req.method+' '+req.path+' from '+req.ip);return false;}
    return true;
  }

  app.get('/api/admin/connection-academy/stats',(req,res)=>{
    if(!requireAdminToken(req,res))return;
    try{
      const db=global.db;
      const total=db.prepare('SELECT COUNT(*) AS c FROM connection_academy').get().c;
      const byStatus=db.prepare('SELECT status, COUNT(*) AS c FROM connection_academy GROUP BY status').all();
      console.log('[OK] /api/admin/connection-academy/stats');
      res.send{success:true,total:total,by_status:byStatus});
    }catch(e){console.error('[ERR] /api/admin/connection-academy/stats:',e.message);res.send({success:false,error:e.message});}
  });

  app.get('/api/admin/connection-academy',(req,res)=>{
    if(!requireAdminToken(req,res))return;
    try{
      const db=global.db;
      const q=(req.query.q||'').trim.toLowerCase();
      const status=(req.query.status||'').trim.toLowerCase();
      const limit=Math.min(+(req.query.limit||50)||50,200);
      const offset=+(req.query.offset||0)||0;
      console.log('[DEBUG] /api/admin/connection-academy q='+q+' status='+status+' limit='+limit+' offset='+offset);
      let sql=`SELECT id,name,email,phone,city,country,interest,message,status,created_at,updated_at FROM connection_academy WHERE 1=1`,arr=[];
      if(q){sql+=` AND (LOWER(name) LIKE ? OR LOWER(email) LIKE ? OR LOWER(phone) LIKE ? OR LOWER(city) LIKE ? OR LOWER(country) LIKE ? OR LOWER(interest) LIKE ? OR LOWER(message) LIKE ?)`;const lq=\`%${q}%\`;arr.push(lq,lq,lq,lq,lq,lq,lq);}
      if(status){sql+=` AND status=?`;arr.push(status);}
      sql+=` ORDER BY id DESC LIMIT ? OFFSET ?`;arr.push(limit,offset);
      console.log('[DEBUG] SQL:',sql,'arr:',arr);
      const rows=db.prepare(sql).all(...arr);console.log('[DEBUG] rows.length=',rows.length);
      console.log('[OK] /api/admin/connection-academy count=',rows.length);
      res.send{success:true,applications:rows,limit:limit,offset:offset});
    }catch(e){console.error('[ERR] /api/admin/connection-academy:',e.message);res.send({success:false,error:e.message});}
  });

  app.patch('/api/admin/connection-academy/:id',(req,res)=>{
    if(!requireAdminToken(req,res))return;
    try{
      const db=global.db;
      const id=parseInt(req.params.id,10);
      if(!Number.isFinite(id)||id<=0)return res.send({success:false,error:'invalid id'});
      const b=req.body||{};console.log('[DEBUG] PATCH body=',b);
      const allowed=['status','notes'];
      const sets=[],arr=[];for(const k of allowed){if(k in b){sets.push(k+'=?');arr.push(b[k]==null?null:(typeof b[k]==='boolean'?(b[k]?1:0):b[k]));}}
      if(sets.length)sets.push('updated_at=datetime(\'now\')');
      arr.push(id);
      const info=db.prepare(`UPDATE connection_academy SET ${sets.join(',')} WHERE id=?`).run(...arr);console.log('[DEBUG] changes=',info.changes);
      console.log('[OK] /api/admin/connection-academy/'+id+' changes='+info.changes);
      res.send{success:true,changes:info.changes});
    }catch(e){console.error('[ERR] /api/admin/connection-academy/:id:',e.message);res.send({success:false,error:e.message});}
  });

  console.log('[READY] connection academy patch loaded');
})();


;; ==== CONNECTION_ACADEMY_AND_DEMO_DATA_END ====

;; ==== CAR_RENTAL_API_ADDITION ====
const fs=require('fs'),path=require('path'),express=require('express');
function requireAdminToken(req,res){
  if(req.ip==='127.0.0.1'||req.ip==='::1')return true;
  const t=(req.header('x-admin-token')||req.query.token||req.body&&req.body.token||'').trim();
  if(!t||t!==(process.env.ADMIN_TOKEN||'').trim){res.status(401).send({success:false,error:'Unauthorized'});console.log('[ADMIN] 401 '+req.method+' '+req.path+' from '+req.ip);return false;}
  return true;
}
const db=global.db;
db.exec(`CREATE TABLE IF NOT EXISTS car_rental_settings (id INTEGER PRIMARY KEY CHECK(id=1),updated_at TEXT,payload TEXT);
CREATE TABLE IF NOT EXISTS car_rental_bookings (id INTEGER PRIMARY KEY AUTOINCREMENT,booking_ref TEXT UNIQUE,created_at TEXT,updated_at TEXT,status TEXT,payment_status TEXT,payment_method TEXT,customer_name TEXT,customer_email TEXT,customer_phone TEXT,pickup_location TEXT,pickup_date TEXT,pickup_time TEXT,return_location TEXT,return_date TEXT,return_time TEXT,car_id TEXT,car_name TEXT,car_class TEXT,daily_rate REAL,rental_days INTEGER,subtotal REAL,insurance REAL,total REAL,currency TEXT,notes TEXT,payload TEXT);`);
console.log('[OK] car_rental_settings + car_rental_bookings ready');
app.get('/api/car-rental/settings',(req,res)=>{try{console.log('[GET] /api/car-rental/settings');const r=db.prepare('SELECT payload FROM car_rental_settings WHERE id=1').get();console.log('[OK] settings',!!r);res.send({success:true,payload:(r&&r.payload)?JSON.parse(r.payload):null});}catch(e){console.error('[ERR] /api/car-rental/settings:',e.message);res.send({success:false,error:e.message});}});
app.post('/api/car-rental/settings',(req,res)=>{try{console.log('[POST] /api/car-rental/settings');if(!requireAdminToken(req,res))return;const p=req.body||{};console.log('[DEBUG] payload keys=',Object.keys(p));db.prepare(`INSERT OR REPLACE INTO car_rental_settings (id,updated_at,payload) VALUES (1,?,?)`).run(new Date().toISOString(),JSON.stringify(p));console.log('[OK] settings saved');res.send({success:true});}catch(e){console.error('[ERR] /api/car-rental/settings:',e.message);res.send({success:false,error:e.message});}});
app.get('/api/car-rental/bookings',(req,res)=>{try{console.log('[GET] /api/car-rental/bookings');if(!requireAdminToken(req,res))return;const rows=db.prepare(`SELECT id,booking_ref,created_at,updated_at,status,payment_status,payment_method,customer_name,customer_email,customer_phone,pickup_location,pickup_date,pickup_time,return_location,return_date,return_time,car_id,car_name,car_class,daily_rate,rental_days,subtotal,insurance,total,currency,notes,payload FROM car_rental_bookings ORDER BY id DESC LIMIT 500`).all();console.log('[OK] bookings count=',rows.length);res.send{success:true,bookings:rows});}catch(e){console.error('[ERR] /api/car-rental/bookings:',e.message);res.send({success:false,error:e.message});}});
app.post/api/car-rental/bookings/:id/status',(req,res)=>{try{console.log('[POST] /api/car-rental/bookings/:id/status');if(!requireAdminToken(req,res))return;const id=req.params.id,{status,payment_status}=req.body||{};console.log('[DEBUG] id=',id,'status=',status,'payment_status=',payment_status);db.prepare(`UPDATE car_rental_bookings SET updated_at=?,status=COALESCE(?,status),payment_status=COALESCE(?,payment_status) WHERE id=?`).run(new Date().toISOString(),status||null,payment_status||null,id);console.log('[OK] updated');res.send{success:true});}catch(e){console.error('[ERR] /api/car-rental/bookings/:id/status:',e.message);res.send({success:false,error:e.message});}});
app.delete('/api/car-rental/bookings/:id',(req,res)=>{try{console.log('[DELETE] /api/car-rental/bookings/:id');if(!requireAdminToken(req,res))return;const id=req.params.id;db.prepare(`DELETE FROM car_rental_bookings WHERE id=?`).run(id);console.log('[OK] deleted');res.send{success:true});}catch(e){console.error('[ERR] /api/car-rental/bookings/:id:',e.message);res.send({success:false,error:e.message});}});
app.get('/api/car-rental/stats',(req,res)=>{try{console.log('[GET] /api/car-rental/stats');if(!requireAdminToken(req,res))return;const r=db.prepare(`SELECT COUNT(*) AS total,SUM(CASE WHEN status='pending' THEN 1 ELSE 0 END) AS pending,SUM(CASE WHEN status='confirmed' THEN 1 ELSE 0 END) AS confirmed,SUM(CASE WHEN status='cancelled' THEN 1 ELSE 0 END) AS cancelled,SUM(CASE WHEN payment_status='paid' THEN total ELSE 0 END) AS revenue_paid,SUM(CASE WHEN payment_status='paid' AND created_at>datetime('now','-7 days') THEN total ELSE 0 END) AS last_7_days_revenue,SUM(CASE WHEN payment_status='paid' AND created_at>datetime('now','-7 days') THEN 1 ELSE 0 END) AS last_7_days_count FROM car_rental_bookings`).get();console.log('[OK] stats');res.send{success:true,stats:r});}catch(e){console.error('[ERR] /api/car-rental/stats:',e.message);res.send({success:false,error:e.message});}});
console.log('[READY] car rental routes ready');

;; ==== CONNECTION_ACADEMY_AND_DEMO_DATA_END ====

;; ==== DEMO_DATA_SEED ====
(function(){
  if (typeof __DEMO_DATA_SEED__) return;
  global.__DEMO_DATA_SEED__ = true;

  const db=global.db;
  console.log('[DEMO] seed start');

  db.exec(`CREATE TABLE IF NOT EXISTS connection_academy (id INTEGER PRIMARY KEY AUTOINCREMENT,name TEXT,email TEXT,phone TEXT,city TEXT,country TEXT,interest TEXT,message TEXT,status TEXT,created_at TEXT,updated_at TEXT);`);

  const cnt=db.prepare('SELECT COUNT(*) AS c FROM connection_academy').get().c;
  console.log('[DEMO] connection_academy count=',cnt);

  if(cnt>=5){console.log('[DEMO] skip, already seeded');}else{
    const ins=db.prepare(`INSERT INTO connection_academy (name,email,phone,city,country,interest,message,status,created_at,updated_at) VALUES (?,?,?,?,?,?,?,?,?,?)`);
    const now=new Date().toISOString();
    const rows=[
      ['Ahmed Mohamed','ahmed.m@demo.com','+20 100 111 2233','Cairo','Egypt','Medical Tourism','Looking for a knee replacement consultation.','pending',now,now],
      ['Sara Khalil','sara.k@demo.com','+20 100 444 5566','Alexandria','Egypt','Dental Tourism','Interested in dental implants and veneers.','contacted',now,now],
      ['Mohamed Ali','mohamed.a@demo.com','+20 100 777 8899','Giza','Egypt','Medical Tourism','Need a second opinion on my MRI results.','pending',now,now],
      ['Nour Hassan','nour.h@demo.com','+20 100 222 3344','Cairo','Egypt','Wellness Tourism','Looking for a wellness retreat package.','pending',now,now],
      ['Khaled Ibrahim','khaled.i@demo.com','+20 100 555 6677','Hurghada','Egypt','Medical Tourism','Cardiac consultation and follow-up care.','contacted',now,now]
    ];
    for(r of rows){ins.run(...r);console.log('[DEMO] inserted',r[0]);}
  }

  console.log('[DEMO] seed done');
})();
```